

Comparative Analysis of Self-Supervised and Supervised Learning Architectures for Image Analysis

(Master's Thesis)

Author: Sarp Koç
sarp.koc@stud.hslu.ch

Supervisor: Dr. Umberto Michelucci
umberto.michelucci@hslu.ch

Co-Supervisor: Dr. Aygul Zagidullina
aygul.zagidullina@hslu.ch

Hochschule Luzern
Master of Science in Applied Information and Data Science (MScIDS)
Autumn Semester 2025

Luzern, 26. November 2025

Contents

1	Introduction	3
2	Literature Review	5
2.1	Historical Overview and Learning Paradigms in Computer Vision	5
2.1.1	Neural Networks and Historical Foundations	5
2.1.2	Convolutional Neural Networks	10
2.1.3	Deep Learning Revolution	14
2.1.4	Transformers and Their Impact on Representation Learning	16
2.1.5	Vision Transformers (ViTs)	20
3	Related Work	27
3.1	Self-supervised Learning	27
3.2	Self-Supervised Learning in Agriculture	31
4	Research Questions / Objectives	34
4.1	Accuracy & Generalization	34
4.2	Label Efficiency	34
4.3	Model Complexity & Efficiency	34
5	Methodology	35
5.1	Dataset	35
5.1.1	Noteworthy Datasets	35
5.1.2	Chosen Dataset	36
5.1.3	Rationale for Dataset Selection	37
5.1.4	Why Classification Task	37
5.2	Data Preparation	38
5.3	Model Architectures	40
5.3.1	Progressive Unfreezing Strategy	40
5.3.2	EfficientNet-B4 (Supervised CNN)	40
5.3.3	DINOv2 (Self-supervised)	41
5.3.4	Vision Transformer (ViT, Supervised Baseline)	41
5.4	Training Framework	41
5.4.1	Training Regimes and Data Availability Scenarios	42
5.4.2	Optimization	43
5.4.3	Progressive Unfreezing and Hyperparameter Variations	44

5.5	Evaluation Protocol	45
5.5.1	Metrics	45
5.5.2	Quantization and Deployment Optimization	46
5.6	Reproducibility	47
5.6.1	Deterministic Training	47
5.6.2	Logging and Checkpointing	47
5.7	Experimental Setup	47
6	Results and Discussion	49
6.1	Results Across Data and Label Fractions	49
6.1.1	Data Scarcity	49
6.1.2	Label Scarcity	53
6.1.3	Extreme Label Scarcity	56
6.1.4	Hyperparameter Sensitivity	57
6.2	Quantization and Deployment Efficiency	61
6.2.1	Quantization - Data Fraction	61
6.2.2	Quantization - Label Fraction	63
6.3	Comparative Discussion	65
7	Challenges	68
8	Conclusion	69
	References	71
9	Declaration of Originality	76
	Declaration of Originality	76

1 Introduction

Plant diseases are one of the most persistent and damaging challenges faced by agricultural systems worldwide. These diseases, triggered by a variety of pathogens and pests, have long hindered food production and imposed severe economic consequences[1]. According to recent studies, up to 40% of global food production is lost each year due to plant diseases and pest infestations[2], [3]. This figure not only highlights the critical importance of crop health but also underscores the fragility of global food systems. The economic losses associated with such agricultural damages are equally staggering, reaching an estimated 220 billion US dollars annually[4].

In response to these challenges, recent advances in digital agriculture and artificial intelligence have opened new possibilities for crop disease monitoring and early detection. Among these, deep learning, a subfield of machine learning, has emerged as a powerful tool in agricultural image analysis. By learning patterns from large volumes of data, deep learning models are capable of performing complex visual recognition tasks that can aid in detecting plant stress, classifying crop types, identifying diseases, and even estimating yield. In particular, Convolutional Neural Networks (CNNs) have been widely adopted due to their ability to extract spatial features from high-resolution images with remarkable accuracy. Models such as VGG, ResNet, and EfficientNet have been applied to a variety of tasks in precision agriculture, including disease detection, weed identification, and crop health assessment[5],[6].

Despite their success, supervised CNNs have a dependency on labeled training data, which can limit scalability in real-world agricultural scenarios. Supervised learning requires expert-annotated images, classified according to disease type and severity. Collecting such datasets is labor-intensive and costly, as it requires specialized botanical and pathological knowledge. Annotation is further complicated by visual ambiguities, overlapping symptoms, and variations in environmental conditions such as lighting, seasonality, and crop variety. Techniques such as fine-tuning pre-trained models (e.g., via transfer learning from ImageNet) and data augmentation, including rotation, scaling, or color transformations, can mitigate these limitations to some extent[5]. Nevertheless, even high-performing architectures such as CNNs and Vision Transformers (ViTs) face challenges when deployed in novel environments. Manual labeling remains expensive and error-prone, especially in domains like remote sensing, where expert knowledge is essential and the majority of imagery remains unlabeled[7], [8].

One promising alternative is self-supervised learning (SSL), a method enabling models to learn meaningful representations from unlabeled data by creating their own supervision via specially designed pretext tasks. Unlike supervised learning, SSL does not require human-labeled examples during the pretraining phase. Instead, the model is trained to predict parts of the input data from other parts, uncovering meaningful structure in the process. Once pre-trained, the model can be fine-tuned on small labeled datasets for specific downstream tasks. This paradigm is particularly attractive in agriculture, where unlabeled imagery from UAVs, satellites, or field sensors is abundant, but expert-labeled data is scarce and costly to obtain[7].

Techniques such as DINO (Self-Distillation with No Labels) and MAE (Masked Autoencoders) enable models, particularly Vision Transformers, to learn powerful visual representations without human-annotated labels[9]. These representations can then be fine-tuned on small labeled datasets for specific tasks. In remote sensing and agricultural applications, SSL-pre-trained ViTs have demonstrated strong

performance in land cover classification, soil mapping, and crop disease detection[8], [10]. Their ability to capture global context and spatial patterns makes them particularly well-suited for UAV-based agricultural imagery. Moreover, SSL models exhibit label efficiency, strong generalization across domains, and robustness in low-label settings, making them ideal candidates for precision agriculture systems[7]. In particular, DINO-pre-trained Vision Transformers demonstrate strong performance in capturing both global context and spatial patterns in aerial imagery, while their flexibility across sensor types and resolutions reduces reliance on costly annotations. Such features become increasingly valuable as UAV technology becomes more accessible, leading to the generation of vast amounts of aerial imagery in real time on farms.

Beyond the fundamental difference in data requirements, the comparative advantages and limitations of self-supervised learning versus supervised learning in agricultural disease detection remain complex and context-dependent. Supervised learning approaches benefit from direct optimization on expert-labeled disease examples, enabling models to learn discriminative features specifically tailored to pathological symptoms and disease progression patterns. When sufficient high-quality labeled data is available, supervised models can achieve precise classification boundaries between healthy and diseased plant tissues, often outperforming other approaches in controlled experimental settings. However, their performance is inherently constrained by the quality, quantity, and representativeness of the training labels[5], making them vulnerable to domain shift when deployed in agricultural environments that differ from their training conditions in terms of crop varieties, environmental factors, or imaging protocols.

In contrast, self-supervised learning models develop representations through exposure to large volumes of unlabeled imagery, potentially capturing more generalizable visual patterns that extend beyond the specific disease categories present in limited labeled datasets. SSL pre-trained models may demonstrate superior robustness when confronted with novel disease presentations, unusual environmental conditions, or imaging artifacts. Their pretraining on diverse unlabeled agricultural imagery may enable them to develop invariant feature representations that generalize better across different crop varieties, growth stages, and imaging conditions. However, this generalization comes with potential trade-offs, as SSL models may lack the specific discriminative power that supervised models gain from direct exposure to expert-annotated disease examples during training[9].

This thesis aims to advance understanding by evaluating the application of both self-supervised and supervised deep learning models for detecting agricultural diseases. While supervised models such as EfficientNet-B4 have achieved high performance in controlled settings with well-labeled data[11], this work examines whether newer self-supervised models, such as DINOv2, can offer competitive or superior results in environments where labeled data is limited. The broader goal is to evaluate whether self-supervised learning can provide a more practical and scalable alternative for deploying AI systems in real-world agricultural contexts. In this thesis, DINO refers to the self-distillation without labels self-supervised learning framework and is used interchangeably with DINOv2, which represents the self-supervised learning approach considered throughout this work.

2 Literature Review

2.1 Historical Overview and Learning Paradigms in Computer Vision

2.1.1 Neural Networks and Historical Foundations

Neural networks (NNs) are a foundational component of modern machine learning, including computer vision. They enable models to learn complex and hierarchical patterns directly from raw data. The idea of mimicking the behavior of the human brain through artificial neurons has evolved over decades, beginning with theoretical concepts and culminating in today's deep learning systems.

McCulloch and Pitts (1943) , Binary Neuron Model. In their foundational paper *A Logical Calculus of the Ideas Immanent in Nervous Activity*[12], Warren McCulloch and Walter Pitts proposed the first mathematical model of a neural network. They described an artificial neuron as a binary threshold unit: it produces an output of 1 if the weighted sum of its inputs exceeds a certain threshold, and 0 otherwise. The model distinguished between excitatory and inhibitory synapses and established a logic-based formalism for neural activity.

McCulloch and Pitts also demonstrated how these binary units could be interconnected to perform logical operations such as AND, OR, and NOT. Their construction used a temporal calculus, allowing neurons to assert propositions about previous states, thus enabling the modeling of sequential processes. This theoretical framework was heavily influenced by propositional logic and symbolic logic, drawing from the *Principia Mathematica*. Although their model lacked any learning capability or weight adaptation, it laid the groundwork for representing cognitive functions in terms of logic circuits and helped establish a bridge between neuroscience and symbolic computation.

This conceptual link between biological and artificial neurons is illustrated in Figure 2.1. On the left, we see a biological neuron with its dendrites, axon, and synapses, while on the right, the simplified artificial neuron mirrors this structure with weighted inputs, summation, and activation functions. This analogy emphasizes how the design of artificial neural networks was inspired by the structural and functional principles observed in neuroscience.

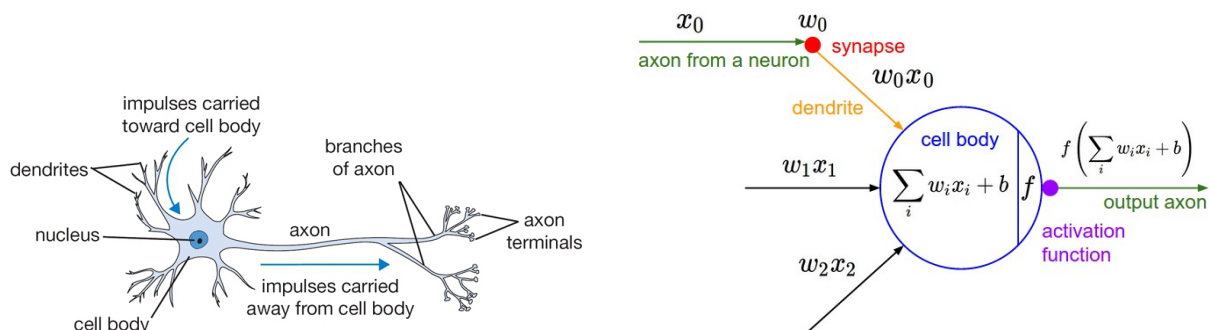


Figure 2.1: Comparison of a biological neuron (left) with a simplified artificial neuron (right). The artificial model mimics key biological mechanisms such as synaptic inputs and activation thresholds, and forms the building block of modern neural networks. Adapted from[13].

Rosenblatt (1958) , The Perceptron. In his 1958 report titled *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*[14], Frank Rosenblatt introduced the concept of the perceptron, a probabilistic model inspired by biological neurons. The perceptron consists of sensory units (input layer), association units (hidden layer), and response units (output layer). It is designed to recognize patterns by modifying the strengths of connections between layers based on a probabilistic learning mechanism. Unlike McCulloch and Pitts' model, which was purely theoretical, perceptron model was implementable, both as an algorithm and as physical hardware.

Rosenblatt formally defined a learning rule whereby the synaptic weights are adjusted when the response output does not match the expected output. He demonstrated that such a system could be trained to respond selectively to specific classes of input patterns. The report also emphasized the biological plausibility of the architecture and provided both theoretical analysis and proposed hardware implementation.

The learning rule of the perceptron is defined by the following update equation:

$$w \leftarrow w + \eta(y_{\text{true}} - y_{\text{pred}})x$$

Here:

- $w \in \mathbb{R}^d$ is the weight vector,
- $\eta \in \mathbb{R}^+$ is the learning rate (a small positive scalar),
- $y_{\text{true}} \in \mathbb{R}$ is the actual label,
- $y_{\text{pred}} \in \mathbb{R}$ is the predicted label,
- $x \in \mathbb{R}^d$ is the input feature vector.

This rule adjusts the weights in the direction that reduces the prediction error. If the prediction is incorrect, the weights are updated proportionally to the input, scaled by the error and the learning rate.

However, the perceptron was only effective for linearly separable problems. It could not model non-linear decision boundaries such as those required for solving the XOR problem¹. For example, even today's linear and multilinear models still fail to solve the XOR problem. Without at least one hidden layer, a neural network simply cannot capture the necessary nonlinearity.

Minsky and Papert (1969) , The XOR Limitation. Minsky and Papert addressed this limitation in their landmark book *Perceptrons*[15], where they mathematically proved that single-layer perceptrons could not solve problems that are not linearly separable, such as XOR. Their analysis focused on simple *feedforward architectures*, networks composed of input and output layers without hidden units. This architecture, used in early neural models, processes data in a one-way flow from input to output but lacks the capacity to model nonlinear decision boundaries.

Although Minsky and Papert acknowledged the theoretical potential of multilayer neural networks, the lack of an effective training algorithm at the time made such architectures quite difficult. Between

¹The XOR (exclusive OR) function outputs 1 only when the two binary inputs differ. Formally, $\text{XOR}(0, 0) = 0$, $\text{XOR}(0, 1) = 1$, $\text{XOR}(1, 0) = 1$, $\text{XOR}(1, 1) = 0$. It is not linearly separable, which means that a single-layer perceptron cannot solve it.

1969 and 1986, there was no widely adopted method for training deep networks, which has become one of the reasons that led to widespread pessimism in the field. As a result, the book also contributed to a significant decline in AI research and funding throughout the 1970s, a period now known as the *first AI winter*[16].

Multilayer Perceptrons (MLPs). The limitations of single-layer perceptrons, as highlighted by Minsky and Papert, motivated researchers to explore architectures with additional layers of computation. These *Multilayer Perceptrons* (MLPs) consist of an input layer, one or more hidden layers, and an output layer. Each neuron in the hidden layers applies a non-linear activation function to a weighted sum of its inputs, enabling the network to learn complex, non-linear decision boundaries.

As we discussed before, the concept of multilayer networks was known as early as Rosenblatt’s work, but neither he nor Minsky and Papert had an effective algorithm for training the hidden layer weights. This limitation meant that MLPs, while theoretically more expressive than single layer perceptrons, remained largely a conceptual tool until the mid-1980s. The breakthrough came with the popularization of the *backpropagation* algorithm, which made it possible to efficiently compute weight updates across multiple layers.

Rumelhart, Hinton, and Williams (1986) , Backpropagation. In 1986, Rumelhart, Hinton, and Williams introduced a general learning procedure for multilayer neural networks called *backpropagation*[17]. This method allowed the network to learn appropriate internal representations by minimizing the error between the actual and desired outputs through gradient descent. While gradient descent itself is a classical optimization technique from calculus, backpropagation made its use in training deep neural networks practical by efficiently computing the gradients through the chain rule.

Unlike earlier methods, backpropagation could train networks with hidden layers, which had previously lacked a mechanism for updating weights. The procedure involved a forward pass to compute outputs and a backward pass to propagate error derivatives layer by layer using the chain rule from calculus. This allowed gradients to be passed backward through the network, enabling the adjustment of weights in all layers, even those not directly connected to the output.

The authors demonstrated that the method could create compact internal representations and generalize well on structured tasks, such as detecting mirror symmetry or inferring family relationships. Although not biologically plausible, the algorithm’s success in constructing distributed internal representations highlighted its importance for learning complex tasks. Backpropagation marked a major turning point by overcoming the limitations of shallow networks, helping to revive interest in neural computation after the stagnation of the 1970s.

The gradient of the loss function L with respect to the weight matrix W is:

$$\frac{\partial L}{\partial W}$$

This expression quantifies how a small change in each weight affects the overall loss (the loss function will be discussed later in this section). These gradients are computed using partial derivatives, typically via the chain rule from calculus. Once the gradients are obtained, the weights are updated using the stochastic gradient descent (SGD) algorithm:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

In this equation:

- η is the learning rate, controlling the step size,
- $\frac{\partial L}{\partial W}$ is the partial derivative of the loss with respect to the weight matrix W ,
- $\leftarrow$ denotes an in-place update of the weight matrix.

Stochastic gradient descent itself was introduced earlier in optimization literature[18], but backpropagation made its practical application to neural network training feasible.

This optimization process is performed iteratively over multiple epochs and data batches, progressively reducing the training error.

Network Architecture and Feedforward Computation. Early neural network models, such as those discussed by Minsky and Papert, employed simple feedforward architectures consisting of an input layer directly connected to an output layer, without any hidden layers. These single-layer perceptrons were limited in their ability to model non-linearly separable functions.

Modern feedforward neural networks extend this structure by introducing one or more *hidden layers* between the input and output layers. Each neuron in a hidden or output layer receives inputs from the previous layer, computes a weighted sum, adds a bias, and applies a non-linear activation function. This architecture allows the network to learn more complex, hierarchical representations.

The linear component of the computation performed by a neuron is given by:

$$z = Wx_i + b$$

where:

- $x_i \in \mathbb{R}^d$ represents the input feature vector for the i -th sample,
- $W \in \mathbb{R}^{k \times d}$ is the weight matrix, where k is the number of neurons in the current layer,
- $b \in \mathbb{R}^k$ is the bias vector,
- $z \in \mathbb{R}^k$ is the resulting vector of pre-activation outputs for the layer.

Note that if the layer contains only a single neuron ($k = 1$), then W reduces to a row vector in $\mathbb{R}^{1 \times d}$, b becomes a scalar, and the output z is also a scalar rather than a vector. This general formulation supports both single and multi-neuron layers.

This linear combination is then passed through a non-linear activation function $f(\cdot)$, yielding the final neuron output:

$$a = f(z) = f(Wx_i + b)$$

Activation Functions and Historical Development. Activation functions introduce non-linearity into neural networks, enabling them to learn complex, non-linear mappings between inputs and outputs. Without such non-linearities, a network composed of multiple linear layers would collapse into a single linear transformation, severely limiting its expressiveness.

The earliest activation function was the binary threshold unit, introduced by McCulloch and Pitts in 1943[12]. As we discussed, in their model, a neuron would “fire” only if the weighted sum of its inputs exceeded a certain threshold:

$$f(x) = \begin{cases} 1 & \text{if } \sum w_i x_i > \theta \\ 0 & \text{otherwise} \end{cases}$$

This binary step function mimicked “all or nothing” firing of biological neurons but lacked differentiability, making it unsuitable for gradient-based optimization methods like backpropagation.

With the emergence of the backpropagation algorithm in the 1980s, researchers required differentiable activation functions. Two major candidates gained prominence during that period:

- **Sigmoid function**, defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

The sigmoid function maps real-valued inputs to the interval (0, 1). It became popular due to its smoothness and differentiability, but it suffers from gradient saturation for large positive or negative values of x , leading to the vanishing gradient problem.

- **Hyperbolic tangent (tanh)** function:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The tanh function maps inputs to the range $(-1, 1)$, and being zero-centered, it often outperformed sigmoid in practice. However, it still experienced saturation issues at extreme values of x .

These early activation functions enabled the practical use of backpropagation in shallow and moderately deep networks. While they are still useful in certain contexts, such as binary classification or gating in recurrent networks, their limitations became more important as models grew deeper in the 2000s and beyond.

A major breakthrough for neural networks came later with the introduction of the Rectified Linear Unit (ReLU) by Glorot, Bordes, and Bengio in 2011[19]:

$$f(z) = \max(0, z)$$

ReLU mitigates the vanishing gradient problem by maintaining a constant gradient for positive values of x . It is computationally efficient and has become the default activation function in modern deep neural networks. Its simplicity and effectiveness contributed significantly to the success of deep learning architectures such as CNNs and ResNets.

Supervised Training and Loss Functions. In supervised learning, a neural network is trained on labeled data to learn a mapping from inputs to corresponding target outputs. As discussed earlier, the model’s predictions are compared to the true labels using a *loss function* L , which quantifies the prediction error.

Two commonly used loss functions are:

- **Mean Squared Error (MSE):** Often used in regression problems, MSE penalizes large errors by squaring the difference between predicted and actual values:

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- y_i is the true label for the i -th sample,
- $\hat{y}_i$ is the predicted output,
- n is the number of samples.

- **Cross-Entropy Loss:** Widely used for classification tasks, particularly when using softmax outputs. It measures the divergence between the predicted probability distribution and the true distribution:

$$L_{\text{CE}} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

- C is the number of classes,
- y_i is the ground truth label (as a one-hot encoded vector),
- $\hat{y}_i$ is the predicted probability for class i .

The goal of training is to minimize the selected loss function over the training set. This is achieved through backpropagation and gradient descent, as previously discussed. The lower the loss, the better the network’s predictions match the actual outputs.

Training proceeds by computing the loss, calculating its gradients via backpropagation, and updating weights using SGD. Over successive iterations, this process allows the model to improve its predictive accuracy.

This enabled, for the first time, the training of neural networks with more than one hidden layer, a critical breakthrough for learning complex, nonlinear functions. Still, progress remained limited for some time due to constraints in computational power and the scarcity of large labeled datasets.

As these limitations gradually diminished, backpropagation became a foundational tool in the development of more sophisticated architectures, including convolutional neural networks (CNNs), that are particularly effective for learning from images, which will be discussed in the following section.

2.1.2 Convolutional Neural Networks

Over time, different architectures and learning paradigms have evolved to address the limitations of early networks. There were numerous attempts to address challenges in visual pattern recognition, laying the foundation for CNNs and similar architectures. One of the earliest influential works came from Hubel and Wiesel in the 1960s in the field of neuroscience, who studied the hierarchical processing of visual

information in the brain, specifically the visual cortex, inspiring the convolutional approach that would later be applied in neural networks[20].

In the 1980s, Kunihiro Fukushima introduced the Neocognitron, a neural network model designed for visual pattern recognition, which featured a hierarchical structure of locally connected units, a concept that directly influenced the architecture of modern CNNs[21]. This work marked a significant departure from fully connected architectures by incorporating local receptive fields, a key feature of CNNs.

These early developments in neural networks set the stage for the formalization of CNNs in 1989[22], marking a critical evolution in the field of machine learning and computer vision. Convolutional Neural Networks (CNNs), first introduced by LeCun and colleagues in 1989, as mentioned, and later popularized with the LeNet-5 architecture in 1998[23], were designed specifically for processing image data and demonstrated impressive performance on tasks such as handwritten digit recognition. By exploiting the spatial structure of images, CNNs reduced the need for extensive preprocessing and achieved better generalization with significantly fewer parameters than fully connected networks.

Comparing Neural Networks and Convolutional Neural Networks. Traditional feedforward neural networks and CNNs share fundamental components such as neurons with associated weights and biases, loss functions for guiding learning, and optimization algorithms like stochastic gradient descent for training. However, their architectures and data handling strategies differ significantly.

In standard feedforward networks, each neuron in a layer is connected to every neuron in the previous layer via learnable weights, forming what is known as a fully connected layer, which will be presented in this section. This dense connectivity leads to a large number of parameters, particularly when dealing with high-dimensional inputs like images. CNNs overcome this inefficiency through architectural innovations tailored for spatial data:

- **Local receptive fields:** Each neuron in a convolutional layer connects only to a small, localized region of the input, allowing the network to focus on local patterns like edges or textures.
- **Weight sharing:** A set of learnable filters (kernels) is convolved across the entire input, meaning the same weights are reused spatially. This greatly reduces the number of trainable parameters.
- **Hierarchical feature extraction:** Stacking multiple convolutional and pooling layers enables the network to learn increasingly abstract features, from edges in early layers to complex shapes and objects in deeper layers.

CNNs also introduce specialized layers such as max pooling (for spatial downsampling), batch normalization (for training stability), and dropout (for regularization). These innovations, combined with their ability to preserve spatial hierarchies, make CNNs especially effective for tasks involving translation invariance and spatial locality, such as image classification, object detection, and semantic segmentation.

This architectural contrast is illustrated in Figure 2.2. On the left, a traditional fully connected network processes 1D feature vectors and ignores spatial relationships. On the right, a convolutional network operates on three-dimensional data (height, width, depth), preserving spatial context throughout the pipeline. This structural advantage enables CNNs to learn rich, spatially-aware representations that are crucial for visual understanding.

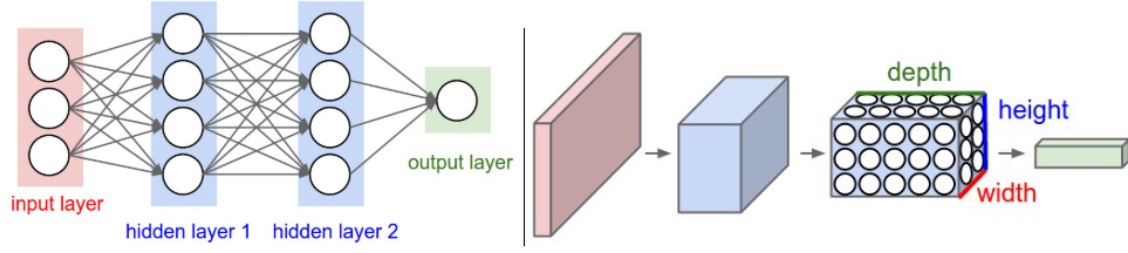


Figure 2.2: Comparison between a traditional fully connected neural network (left) and a convolutional neural network (right). While fully connected networks process 1D feature vectors, convolutional networks operate on 3D volumes (width, height, depth), preserving spatial structure. In this example, the input volume depth is 3, corresponding to RGB channels. Adapted from[13].

Mathematical Formulation and Key Components of CNNs Convolutional Neural Networks (CNNs) introduced several architectural innovations that make them well suited for image data. The three main layer types used to construct CNNs are *convolutional*, *pooling*, and *fully connected* layers[13]. Each layer transforms a 3D input volume into a 3D output volume via a differentiable function, leveraging spatial structure and reducing parameters.

Convolution Operation. Given an input volume $I \in \mathbb{R}^{H \times W \times C}$ and a filter (kernel) $K \in \mathbb{R}^{k \times k \times C}$, each convolution computes at location (i, j) :

$$S(i, j) = \sum_{m=1}^k \sum_{n=1}^k \sum_{c=1}^C I(i+m-1, j+n-1, c) \cdot K(m, n, c)$$

Here, the filter slides spatially over the input, yielding a feature map. This local receptive field mechanism means each neuron connects only to a small spatial region, with weights shared across all locations, drastically reducing parameter count compared to fully connected layers[13].

Spatial Output Size and Receptive Fields. The spatial dimensions of the output volume in a convolutional layer depend on the input size W , kernel size K , zero-padding P , and stride S :

$$W_{\text{out}} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

This formula determines how many positions the kernel can slide across the input volume, thereby computing the width and height of the output feature map. For example, increasing the stride S reduces the output size, while adding more padding P can help preserve the spatial resolution.

Each additional convolutional layer increases the effective receptive field, the region of the input that a particular neuron is sensitive to. After L layers with kernel size K (assuming no downsampling or dilation), the receptive field size becomes:

$$\text{RF} = 1 + L \cdot (K - 1)$$

This shows that deeper layers have a larger “field of view” in the input, allowing them to recognize higher-level spatial patterns[13].

Pooling Layers. Pooling layers (commonly max pooling) reduce spatial dimensions using operations such as non-overlapping 2×2 windows with stride 2. They increase translation invariance, reduce computation, and help prevent overfitting without any learnable parameters[13]. While max pooling selects the maximum value in each region, other pooling methods such as average pooling (or mean pooling) compute the mean, and are occasionally used depending on the task.

Fully Connected Layers. At the end of a convolutional neural network, the final feature maps are typically flattened into a one-dimensional vector and passed through one or more fully connected (dense) layers. These layers integrate the high-level spatial features extracted by the convolutional layers and transform them into class scores for prediction, similar to traditional neural networks. This final stage enables the network to perform tasks such as classification or regression based on the learned representation[13].

Convolutional neural networks gained prominence due to several important developments over time. The availability of large, well-annotated datasets laid the foundation for effective training, while advancements in GPU technology provided the necessary computational power to handle their complexity. In parallel, innovations in regularization methods, such as dropout, significantly enhanced model generalization and robustness.

Together, these advances enabled CNNs not only to exploit the spatial structure of images efficiently but also to learn hierarchical feature representations in a data driven manner. Unlike traditional fully connected networks, CNNs achieve this with far fewer parameters, making them both computationally efficient and highly scalable. These breakthroughs have played a crucial role in the widespread adoption and ongoing success of CNNs since their original introduction[24].

However, despite their success, CNNs also have limitations, which have motivated the exploration of alternative architectures, such as Vision Transformers (ViT), which will be discussed in more detail in the following section, along with a more in-depth look at these limitations. For this section, some of these limitations include:

- **Limited receptive field:** CNNs rely on local receptive fields, which can struggle to capture long-range dependencies and global context in images, often requiring deeper networks to progressively aggregate features.
- **Data inefficiency:** CNNs often require large amounts of labeled data to perform well, particularly for tasks that involve complex visual relationships or intricate details.
- **Translation invariance:** CNNs are not inherently invariant to translations, rotations, or other transformations of objects in an image, often necessitating additional techniques like data augmentation.
- **Fixed spatial hierarchy:** While the hierarchical structure of CNNs is useful for capturing local patterns, it may not always be the most effective for modeling global relationships across an image.

In the following section, we will explore some of the key deep learning breakthroughs that paved the way for Vision Transformers (ViTs), marking a significant shift in the approach to computer vision.

2.1.3 Deep Learning Revolution

Following the revival of neural networks in the 1980s through the development of backpropagation, advances in hardware (GPUs), large-scale datasets, and optimization techniques set the stage for the modern era of deep learning, which refers to neural architectures composed of many successive layers, often called *deep* networks. These deep architectures are capable of learning increasingly abstract and hierarchical representations of data, making them especially effective for complex computer vision tasks such as object detection, semantic segmentation, and image generation.

A pivotal moment for deep learning came with the introduction of AlexNet by Krizhevsky et al.[25], which won the 2012 ImageNet Large Scale Visual Recognition Challenge (ILSVRC). Its success was fueled by the availability of the large-scale ImageNet dataset containing 1.2 million labeled images[26], the use of GPU-accelerated training for efficient computation[25], and the application of dropout at scale for regularization[27]. AlexNet demonstrated that deep convolutional neural networks trained on GPUs could significantly outperform traditional methods when large-scale labeled datasets were available. Its architecture introduced key innovations such as:

- The use of ReLU activation functions $f(x) = \max(0, x)$ instead of saturating nonlinearities like sigmoid,
- Overlapping max-pooling to reduce spatial dimensions while retaining important features,
- Dropout regularization[27] to prevent overfitting.

Dropout. Dropout is a simple yet powerful technique that reduces overfitting by randomly “dropping” units during training². Specifically, each neuron’s output is set to zero with probability $1 - p$, where $p \in [0, 1]$ is the probability of retaining the unit. The forward-pass during training can be expressed as:

$$h'_i = r_i \cdot h_i \quad \text{where} \quad r_i \sim \text{Bernoulli}(p)$$

- h_i is the activation of neuron i ,
- $r_i \in \{0, 1\}$ is a randomly sampled binary mask,
- h'_i is the resulting masked activation passed to the next layer.

During inference, dropout is disabled and the weights are scaled by p to maintain the expected output magnitude:

$$\mathbb{E}[h'_i] = p \cdot h_i$$

This technique can be interpreted as training an ensemble of exponentially many subnetworks and averaging their predictions at test time, providing a form of model averaging that improves generalization.

²Note that some modern deep learning frameworks, such as PyTorch, use inverted dropout, where activations are scaled during training rather than inference, so that no adjustment is needed at test time.

Following AlexNet, the VGG network by Simonyan and Zisserman in 2014[28] demonstrated that increasing depth while using small 3×3 convolutional filters could significantly improve accuracy. VGG employed a simple, uniform design: stacking multiple convolutional layers with ReLU activations, followed by max-pooling and fully connected layers. This straightforward architecture became a popular baseline in research and a widely used feature extractor for transfer learning. However, VGG’s high computational and memory costs limited its practicality, motivating the development of more efficient deep network designs.

Moreover, simply increasing the number of layers introduced new challenges, especially the degradation problem, where deeper networks began to perform worse than shallower counterparts due to vanishing gradients and optimization difficulties. This issue was overcome by the introduction of ResNet (Residual Networks) by He et al.[29] in 2015, which proposed *skip connections* (or *identity shortcuts*) that allowed gradients to flow more effectively through the network:

$$y = \mathcal{F}(x, \{W_i\}) + x$$

Here, $\mathcal{F}(x, \{W_i\})$ represents the residual mapping to be learned, and the input x is directly added to the output of the residual block. This architectural change enabled networks with over 100 layers to be trained effectively and laid the foundation for modern vision backbones. In addition to mitigating vanishing gradients, ResNet addressed optimization difficulties by reformulating the learning task as fitting residual mappings rather than full transformations, allowing deeper networks to converge more effectively.

The degradation problem. The degradation problem refers to the phenomenon where adding more layers to a neural network results in higher training and test errors, even though additional layers should, in principle, be able to learn an identity mapping and at least match the performance of a shallower model. This issue is *distinct from overfitting*: while overfitting occurs when a model performs well on training data but poorly on unseen data, degradation affects performance even on the training set. It is often caused by optimization difficulties such as vanishing gradients, where the magnitude of gradients decreases exponentially as they are backpropagated through many layers, making weight updates in early layers extremely small and slowing or halting learning, and exploding gradients, where gradients grow exponentially during backpropagation, causing unstable parameter updates, numerical overflow, and divergence during training.

Formally, if we denote the loss function by $\mathcal{L}$, the weight matrix of layer l by $W^{(l)}$, and the activation at layer L by $h^{(L)}$, the gradient with respect to the first layer’s weights can be expressed as:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \left(\prod_{l=2}^L W^{(l)} \right) \cdot \frac{\partial \mathcal{L}}{\partial h^{(L)}}$$

If the norms of $W^{(l)}$ are less than 1, repeated multiplication drives the gradient magnitude toward zero (vanishing gradients). Conversely, if the norms are greater than 1, the gradient magnitude grows exponentially (exploding gradients).

Lastly, while most early deep learning efforts focused on supervised learning, neural networks have

since been adapted to unsupervised, semi-supervised, and self-supervised learning paradigms³. Examples include:

- **Autoencoders**, which learn to reconstruct inputs and capture latent representations.
- **Generative Adversarial Networks (GANs)**, composed of a generator and a discriminator, trained in a minimax game:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

- **Self-supervised learning**, where models are trained using pretext tasks that require no manual labels (e.g., contrastive learning, masked image modeling, or clustering-based methods such as DINO with a teacher, student architecture, covered in detail in section 3.1 Self-supervised Learning).

In recent years, deep learning has increasingly shifted toward transformer-based architectures, such as the Vision Transformer (ViT)[30], which leverage self-attention mechanisms to model long-range dependencies in images. ViT and its variants have achieved state-of-the-art performance on numerous computer vision benchmarks and have become a dominant paradigm for large-scale vision tasks. These advances in deep learning are not only very important to modern computer vision, but also explain why this part of the history is included here: understanding how deep architectures evolved, and the challenges they overcame, helps clarify why many of the principles and techniques developed in deep learning continue to play a crucial role, even in transformer-based models. The next section examines these transformer architectures in detail, followed by a closer look at the Vision Transformer and self-supervised learning methods that build upon them.

2.1.4 Transformers and Their Impact on Representation Learning

While convolutional neural networks have remained the backbone of computer vision for many tasks, the past few years have also seen the rise of a new family of architectures in NLP field: Transformers. First introduced by Vaswani et al.[31] for sequence modeling in natural language processing (NLP), particularly in translation tasks, the Transformer architecture replaced recurrence with a mechanism called self-attention, enabling models to capture dependencies between elements, regardless of their position. This capability of modeling long-range relationships with greater flexibility than traditional architectures has driven interest in adapting Transformers beyond NLP tasks.

For example, the ability of Transformers to model global context in a single layer is particularly valuable in self-supervised learning settings, where the objective often involves relating distant parts of the input, such as matching widely separated image patches in computer vision or reconstructing masked regions. This global receptive field allows the model to capture high-level semantic relationships early in the processing pipeline, reducing the need for progressively aggregating local features, as seen in CNNs. These aspects will be examined in a much more detailed way in that section.

³*Unsupervised learning*: The model is trained entirely on unlabeled data to discover patterns or groupings (e.g., clustering, dimensionality reduction). *Semi-supervised learning*: The training set contains a small labeled subset and a large unlabeled subset; the model leverages both to improve performance. *Self-supervised learning*: The model uses only unlabeled data, with supervision provided by automatically generated labels through pretext tasks such as contrastive learning or masked modeling.

Transformer Architecture. The original Transformer is composed of an encoder-decoder structure. The encoder maps an input sequence of symbol representations $x = (x_1, \dots, x_n)$ to a sequence of continuous representations $z = (z_1, \dots, z_n)$, while the decoder generates an output sequence $y = (y_1, \dots, y_m)$ autoregressively.

Each encoder and decoder layer consists of two main sub-layers:

- A multi-head self-attention mechanism
- A position-wise feed-forward neural network (FFN)

Additionally, residual connections and layer normalization are applied after each sub-layer.

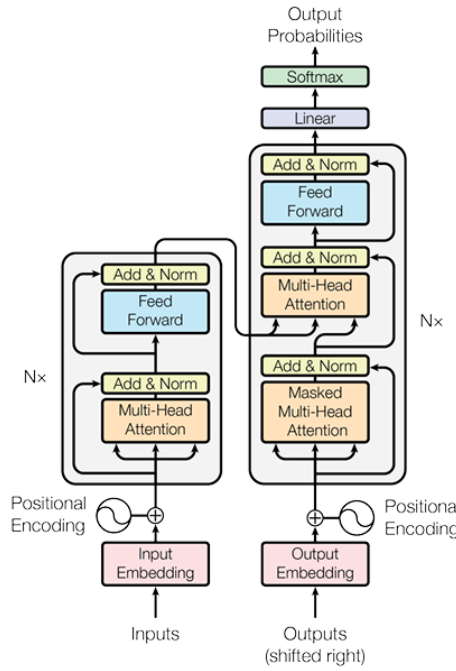


Figure 2.3: Transformer model architecture. Adapted from [31].

Multi-Head Attention. Instead of applying a single attention mechanism, the Transformer uses multi-head attention, which allows the model to jointly attend to information from different representation subspaces. Each head computes attention in parallel, and the outputs are concatenated and linearly transformed:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where each attention head is defined as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Here, W_i^Q, W_i^K, W_i^V are learned projection matrices for the i th head, and W^O is the final output projection matrix.

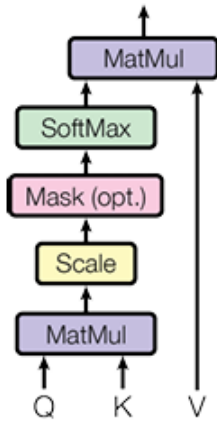
The attention mechanism inside each head is implemented using scaled dot-product attention. Given

queries Q , keys K , and values V , the attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

The dot products between queries and keys are divided by $\sqrt{d_k}$, where d_k is the dimensionality of the key vectors. This scaling helps prevent excessively large dot-product values, which can push the softmax function into regions with very small gradients, leading to unstable learning. The resulting attention scores are used to compute a weighted sum of the value vectors.

Scaled Dot-Product Attention



Multi-Head Attention

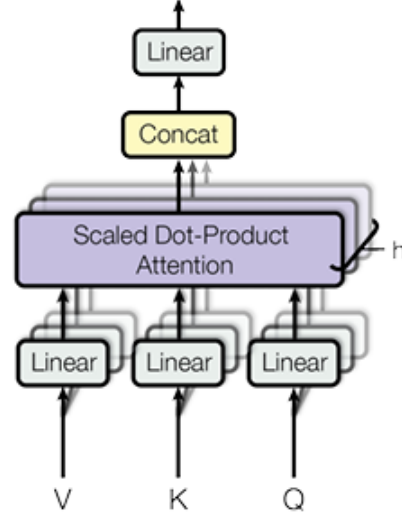


Figure 2.4: On the left, Scaled Dot-Product Attention computes attention weights based on the similarity between queries and keys. On the right, Multi-Head Attention runs several such attention mechanisms in parallel, enabling the model to capture diverse contextual relationships.. Adapted from[31].

Feed-Forward Networks and Positional Encoding. Each layer also includes a feed-forward network applied independently to each position:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Since the attention mechanism is permutation-invariant, Transformers use positional encodings to inject information about the relative or absolute position of tokens in the sequence. Vaswani et al. proposed sinusoidal encodings, though learned positional embeddings have also been widely adopted.

Advantages. Transformers are highly parallelizable compared to recurrent models, since they do not require sequential processing during training. This enables faster training times and better scalability on modern hardware, especially GPUs and TPUs. Additionally, the self-attention mechanism allows each token to directly attend to all others in a sequence, enabling the model to capture long-range dependencies more effectively than RNNs or CNNs.

Another advantage is the architectural modularity of Transformers. Each layer contains the same

core building blocks (attention + feed-forward layers), allowing for deep stacking and straightforward scaling to hundreds of layers or billions of parameters. This flexibility has facilitated the development of large-scale pre-trained models that can be fine-tuned for various downstream tasks.

Transformers also support various pretraining strategies, such as masked language modeling (used in BERT) or causal language modeling (used in GPT), enabling models to learn contextual representations from vast unlabeled corpora. This transfer learning capability has been a major driver of their widespread adoption across NLP and beyond.

Finally, the attention mechanism provides a form of model interpretability. Attention weights can be visualized to understand which input elements influence the model’s decisions, though this interpretability is not without limitations.

Decoder Masking in Causal Transformers. Decoder-only architectures, such as GPT[32], generate output tokens autoregressively. To prevent the model from attending to future tokens (which would leak information during training), a technique called causal masking is used. Specifically, a triangular mask is applied to the attention scores before the softmax operation, ensuring that each token can only attend to itself and preceding tokens in the sequence.

Mathematically, the mask M is applied as follows:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V$$

where $M_{ij} = -\infty$ for $j > i$, and 0 otherwise. This mask zeroes out future positions by assigning them large negative scores, effectively removing them from the attention computation. During inference, tokens are generated one-by-one, maintaining the left-to-right causal structure.

Architectural Variants. Transformer models can be grouped into three broad architectural categories, each suited to different types of tasks:

- **Encoder-only Transformers** (e.g., BERT[33]): These models process input sequences in full using bidirectional self-attention. They are well-suited for tasks such as classification, sentence embedding, or question answering, where contextual understanding of the input is needed.
- **Decoder-only Transformers** (e.g., GPT[32]): These models use causal self-attention and generate tokens autoregressively. They are typically used for generative tasks like text completion, summarization, or dialogue generation.
- **Encoder-decoder Transformers** (e.g., T5[34]): Also known as sequence-to-sequence models, they use an encoder to process the input and a decoder to generate the output. They are ideal for tasks like machine translation, summarization, or image captioning, where input and output are distinct sequences.

Modern Industrial Applications. While the variants discussed above represent important research contributions, contemporary large-scale models deployed by industry leaders often employ more sophisticated hybrid architectures. Modern systems like GPT-4 and Claude typically integrate Mixture of Experts (MoE) architectures[35], which activate only subsets of parameters per input, dramatically improving computational efficiency while maintaining model capacity. These models also incorporate

custom attention mechanisms, advanced training techniques such as reinforcement learning from human feedback (RLHF)[36], and specialized optimizations for distributed computing infrastructure. However, the specific architectural details of these commercial systems remain largely proprietary, representing a growing gap between publicly available research and state-of-the-art deployed models.

4

Transformers Beyond NLP. As we have already talked in previous section, the success of Transformers in NLP sparked interest in applying similar architectures to other domains, particularly computer vision. Traditional convolutional neural networks rely on local receptive fields and translation invariance, which are highly effective for image tasks. However, they also have limitations in modeling global dependencies.

Researchers began exploring whether the attention-based modeling power of Transformers could be leveraged in vision tasks. This led to the development of the Vision Transformer (ViT) by Dosovitskiy et al.[30], which directly applies Transformer encoders to sequences of image patches instead of word tokens.

2.1.5 Vision Transformers (ViTs)

The overwhelming success of Transformers in natural language processing naturally led researchers to investigate whether similar attention-based architectures could revolutionize computer vision. For over a decade, convolutional neural networks (CNNs) had dominated visual recognition tasks due to their inherent inductive biases: translation equivariance, local connectivity, and weight sharing. However, these biases also impose limitations on modeling long-range spatial dependencies and global context, which are crucial for understanding complex visual scenes.

Early Attempts and Motivation. Initial efforts to incorporate attention mechanisms into vision models focused on augmenting CNNs rather than replacing them entirely. Capsule Networks (CapsNets), proposed by Geoffrey Hinton and colleagues[37], aimed to address some of the limitations of CNNs, particularly viewpoint invariance and the ability to generalize better to new viewpoints. Unlike CNNs, which use convolutions and pooling to focus on local features in an image (such as edges or textures), CapsNets take a different approach. Instead of just focusing on individual pixels or small regions, CapsNets use "capsules", groups of neurons that work together to capture complex information about the object.

Each capsule is designed to detect specific patterns or parts of an object and preserve the spatial relationship between those parts. This means that a capsule can understand how different parts of an object (e.g., the top, bottom, or side of a car) are positioned in relation to each other, even if the object moves, rotates, or changes size. For example, if a car turns or rotates, the capsule network can still recognize it as a car because it understands the spatial relationships between the parts (wheels, doors, etc.), even when the appearance of those parts changes due to the rotation.

This is different from CNNs, where parts of the object might become distorted or misaligned when rotated or transformed, making it harder to recognize the object correctly. By preserving the spatial relationships between parts, CapsNets are designed to be more robust to changes like rotation, translation,

⁴Although academic research on techniques like MoE and RLHF is publicly available, commercial implementations such as GPT-4 and Claude often use proprietary adaptations combined with undisclosed infrastructure and scaling strategies.

or scaling, making them better suited for certain tasks that involve such transformations.

However, despite their theoretical advantages, CapsNets faced significant challenges in practical applications:

- **Computational Complexity:** The dynamic routing algorithm used in CapsNets is computationally expensive and inefficient, particularly for large datasets, which can make training more resource-intensive compared to CNNs[37]. In contrast, CNNs address the problem of transformation invariance through data augmentation, which allows them to generalize to various transformations without the need for complex, computationally expensive mechanisms[5].
- **Scalability Issues:** CapsNets struggled to scale to larger datasets and tasks, as they did not consistently outperform CNNs on large-scale benchmarks such as ImageNet[26].
- **Training Difficulties:** The dynamic routing mechanism, while innovative, made training unstable and difficult, causing challenges in practical implementation.

These limitations, combined with the already strong performance and efficiency of CNNs, led to the wider adoption of CNNs.

However, the breakthrough came with the observation that if Transformers could process sequential data effectively, images could potentially be treated as sequences of patches. This insight challenged the fundamental assumption that convolution was necessary for visual understanding and opened the door to pure attention-based vision models.

Vision Transformer Architecture. In 2020, Dosovitskiy et al.[30] introduced the Vision Transformer (ViT), which directly applies the standard Transformer encoder architecture to image classification. The key innovation lies in how images are preprocessed and tokenized for the attention mechanism.

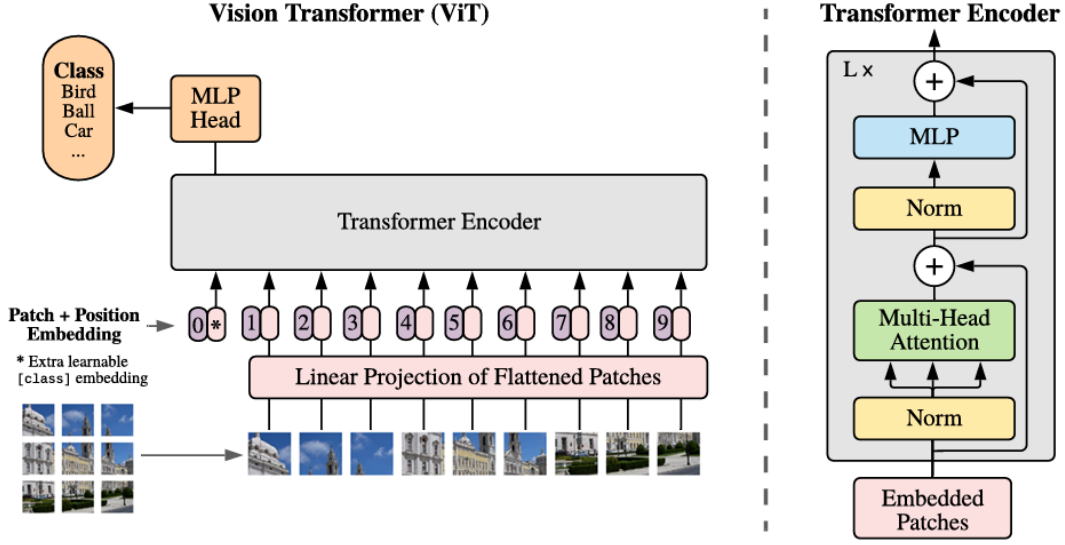


Figure 2.5: This figure illustrates the Vision Transformer (ViT) model architecture. The model works by dividing an image into fixed-size patches, embedding each patch into a vector, and adding position embeddings. These vectors are then passed through a Transformer encoder. To perform classification, a learnable "classification token" is appended to the sequence. This illustration of the Transformer encoder is inspired by the work of[31], and the methodology is adapted from[30].

Given an input image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H and W are the height and width, and C is the number of channels, ViT divides the image into a sequence of non-overlapping patches. Each patch has dimensions $P \times P$, resulting in $N = HW/P^2$ patches. The sequence of patches is then flattened and linearly projected into a D -dimensional embedding space:

$$\mathbf{x}_p^{(i)} = \text{Flatten}(\mathbf{x}_{patch}^{(i)}) \cdot \mathbf{E}$$

where $\mathbf{E} \in \mathbb{R}^{P^2 \cdot C \times D}$ is a learnable projection matrix, and $\mathbf{x}_p^{(i)}$ represents the i -th patch embedding.

Similar to BERT, ViT prepends a learnable classification token $\mathbf{x}_{class}$ to the sequence of patch embeddings. This token serves as a global representation that aggregates information from all patches through self-attention. The final sequence becomes:

$$\mathbf{z}_0 = [\mathbf{x}_{class}; \mathbf{x}_p^{(1)} \mathbf{E}; \mathbf{x}_p^{(2)} \mathbf{E}; \dots; \mathbf{x}_p^{(N)} \mathbf{E}] + \mathbf{E}_{pos}$$

where $\mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$ represents learnable positional embeddings that encode spatial information about patch locations.

The embedded patches are then processed through L layers of standard Transformer encoders. Each layer ℓ applies multi-head self-attention (MSA) and a feed-forward network (MLP) with residual connections and layer normalization:

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1} \quad (2.1)$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell \quad (2.2)$$

Finally, the classification head consists of a simple MLP applied to the final representation of the classification token:

$$\mathbf{y} = \text{MLP}(\text{LN}(\mathbf{z}_L^{(0)}))$$

where $\mathbf{z}_L^{(0)}$ is the classification token's representation after the final Transformer layer.

Experimental Findings and Scaling Behavior. ViT's initial results were both promising and revealing about the scaling behavior of attention-based vision models. When pre-trained on large datasets such as ImageNet-21k (14 million images) or JFT-300M (300 million images), ViT achieved competitive or superior performance compared to state-of-the-art CNNs like ResNet and EfficientNet on standard benchmarks including ImageNet, CIFAR-10, and CIFAR-100[30].

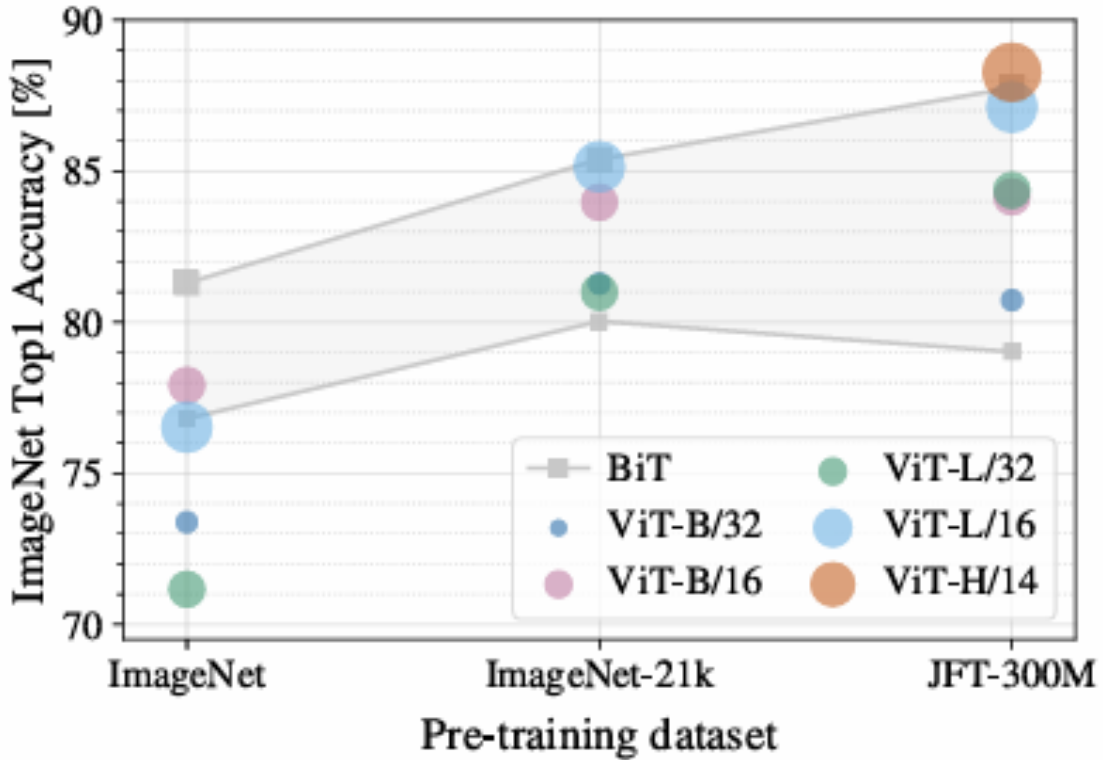


Figure 2.6: Performance of ViT models on ImageNet transfer learning. While larger ViT models initially underperform compared to BiT ResNets (indicated by the shaded area) when pre-trained on smaller datasets, they significantly outperform as the dataset size increases. Additionally, as the dataset grows, larger ViT models surpass smaller ones in performance. This figure is adapted from[30].

Crucially, as it has been shown, ViT demonstrated strong scaling properties with respect to both data

and model size. Unlike CNNs, which often plateau in performance with increased data, ViT continued to improve as the training dataset size grew. This suggested that the global modeling capacity of self-attention could better leverage large-scale data compared to the local inductive biases of convolution.

However, when trained on smaller datasets like ImageNet-1k (1.3 million images) from scratch, ViT underperformed ResNet-based models, as it has been discussed. This highlighted a key limitation: ViT requires substantial amounts of training data to learn effective spatial relationships without the built-in inductive biases of CNNs.

Subsequent Improvements and Variants. The success of ViT sparked rapid development of improved Transformer-based vision architectures. Touvron et al.[38] in 2021 introduced Data-efficient Image Transformers (DeiT), which addressed ViT’s data inefficiency through advanced training strategies, including teacher-student distillation and extensive data augmentation. DeiT demonstrated that ViTs could achieve competitive performance even when trained on smaller datasets like ImageNet-1k. While DeiT is a supervised method, another important advance in transformer-based vision models was the introduction of DINO (2021) (Distillation with No Labels)[39], a self-supervised approach that also uses teacher-student distillation techniques but without requiring labeled data. We will discuss this approach and its techniques in more detail later, as it is also the main architecture DINOv2 (2023)[40] used in this thesis.

A significant advancement came with Swin Transformers[41], introduced by Liu et al. (2021), which introduced hierarchical processing and shifted window attention. Unlike ViT’s fixed-size patches, Swin Transformers process images at multiple scales using a pyramid architecture similar to CNNs. The shifted window mechanism computes attention within local windows and shifts these windows across layers, enabling efficient computation while maintaining global connectivity:

W-MSA : Attention within windows of size $M \times M$

SW-MSA : Shifted windows by $\lfloor M/2 \rfloor$ pixels

This design reduces computational complexity from quadratic to linear with respect to image size while enabling cross-window connections through the shifting mechanism.

Another important variant, the Pyramid Vision Transformer (PVT)[42], extends ViT by incorporating pyramid structures to handle dense prediction tasks such as segmentation and object detection. The PVT introduces multi-scale processing, enabling more efficient and accurate feature extraction, especially in tasks requiring pixel-level predictions.

Additionally, the Twins model[43] combines local and global attention mechanisms to improve computational efficiency. These architectural advancements, along with others, highlight the growing versatility of Transformer models in vision tasks, influencing the development of models like discussed DINOv2, which achieves both efficiency and performance improvements.

Theoretical Analysis and Understanding. Recent theoretical work has provided insights into why ViTs work and how they differ from CNNs. Raghu et al.[44] analyzed the representation learning dynamics of ViTs, showing that they learn more uniform representations across layers compared to CNNs, which develop increasingly specialized features in deeper layers.

Park and Kim[45] demonstrated that ViTs can learn spatial relationships through positional embed-

dings and attention patterns, effectively reconstructing spatial locality that CNNs encode through convolution. This explains why ViTs require more data, they must learn spatial biases that are hardcoded into CNNs.

Computational Considerations and Efficiency. While ViTs demonstrate excellent performance on large datasets, their computational requirements present challenges for practical deployment. The self-attention mechanism has $O(N^2)$ complexity with respect to the number of patches N , making it expensive for high-resolution images. Because this quadratic complexity becomes increasingly costly when dealing with high-resolution images, as the number of patches grows significantly with resolution.

Big-O notation. In computer science, we use Big-O notation, also called Landau’s symbol, to describe the performance or complexity of an algorithm, particularly how it scales with the size of the input. The notation provides an upper bound on the running time or space requirements of an algorithm. For example, $O(N^2)$ means that as the size of the input, N , grows, the time or space required by the algorithm increases quadratically. So, if the input doubles, the resource usage will quadruple. Common complexities include $O(1)$, which indicates constant time (the algorithm takes the same time regardless of input size), and $O(N \log N)$, which is more efficient than $O(N^2)$ for large inputs. Similarly, $O(N)$ represents a linear increase in time or space as the input grows. These complexities help developers compare and choose algorithms that are best suited for the problem at hand, considering both the size of the input and the desired efficiency[46].

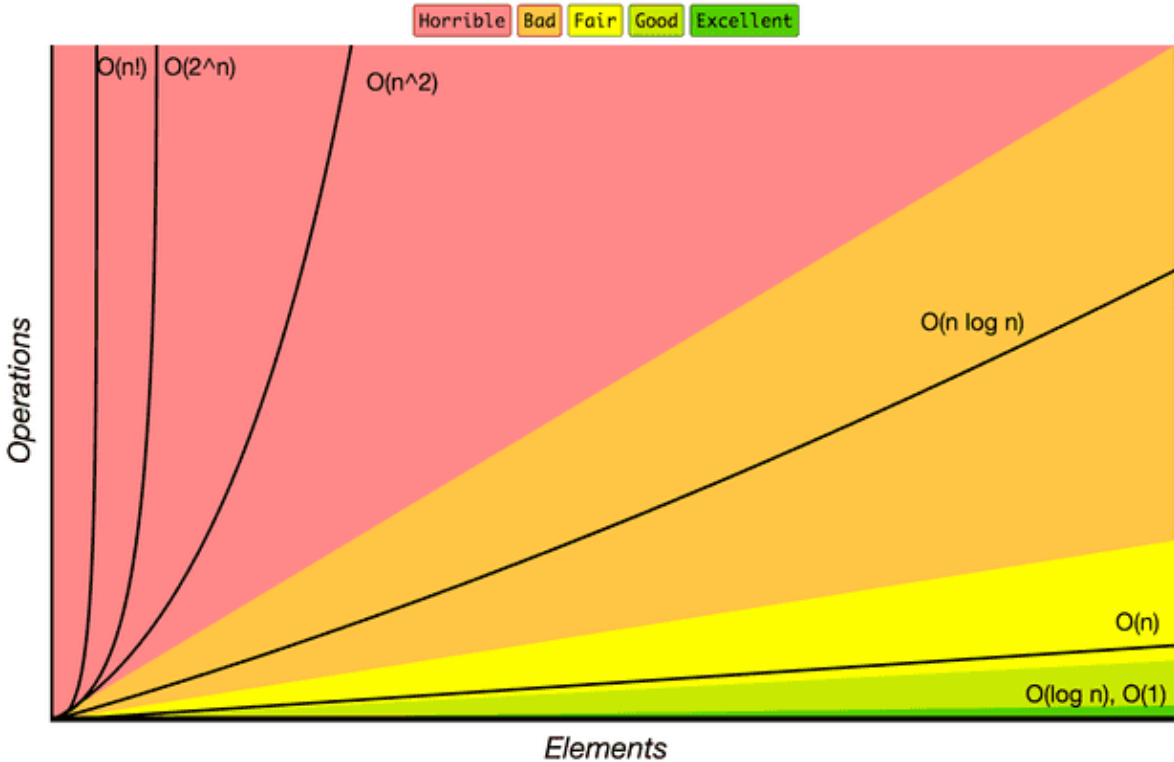


Figure 2.7: Common time complexities: $O(1)$ (Constant), $O(\log n)$ (Logarithmic), $O(n)$ (Linear), $O(n \log n)$ (Linearithmic), $O(n^2)$ (Quadratic), $O(2^n)$ (Exponential), and $O(n!)$ (Factorial)[47].

Several approaches have been proposed to address this limitation. Hierarchical architectures like Swin Transformers process images at multiple resolutions, concentrating computation on the most in-

formative regions. Hybrid models combine convolutional layers for early feature extraction with Transformer layers for global reasoning, leveraging the strengths of both architectures[48], [49].

Applications Beyond Image Classification. The success of ViTs in image classification has extended to numerous other vision tasks. Object detection has been addressed by Detection Transformer (DETR) (2020)[50], which formulates detection as a set prediction problem using Transformers. Semantic segmentation benefits from ViTs’ global modeling capabilities, with architectures like SETR (2021)[51] achieving strong performance by treating segmentation as a sequence to sequence translation task.

ViTs have also shown promise in video understanding, where their ability to model long-range temporal dependencies complements their spatial modeling capabilities. TimeSformer (2021)[52] and Video Vision Transformer (ViViT) (2021)[53] extend ViT to video by incorporating spatiotemporal attention mechanisms. These models enable ViTs to effectively handle the temporal dimension in videos, paving the way for state-of-the-art performance in action recognition and other video-related tasks.

Current Limitations and Future Directions. Despite their success, ViTs face several ongoing challenges. Their dependence on large datasets limits applicability to domains with scarce data such as agriculture. Computational requirements remain high[2], [7], [10], especially for high-resolution images or real-time deployment scenarios.

To address this limitation, self-supervised learning (SSL) approaches are gaining increasing attention[54]. By learning useful representations from unlabeled data, self-supervised models can reduce the need for large labeled datasets, making them particularly attractive for domains with limited annotated data. The next section, we will cover SSL approaches.

3 Related Work

Self-supervised learning (SSL) in computer vision has rapidly evolved over the past decade, with numerous algorithmic directions explored. This section highlights a subset of influential approaches, particularly in contrastive learning, clustering-based methods, masked image modeling, and self-distillation with Vision Transformers (ViTs) that form the methodological background for DINOv2.

3.1 Self-supervised Learning

Teacher-Student Ideas The teacher-student paradigm originates from supervised *knowledge distillation*[55], where a smaller *student* model is trained to reproduce the output distribution of a larger, more complex *teacher* model. This idea builds on earlier work by Caruana and collaborators[56], who demonstrated that the knowledge contained in a cumbersome ensemble can be transferred into a single, smaller model that is easier to deploy.

Hinton et al.[55] formalized this process by introducing the concept of *soft labels*: instead of using only one-hot hard labels, the student is trained to match the teacher’s softened probability distribution, obtained by dividing the teacher’s logits $\mathbf{z}^{(t)}$ by a temperature $T > 1$:

$$q_i^{(t)} = \frac{\exp(z_i^{(t)}/T)}{\sum_k \exp(z_k^{(t)}/T)}.$$

The student predicts its own softened distribution $p_i^{(s)}$ in the same way, and the distillation loss is defined as:

$$\mathcal{L}_{\text{KD}} = T^2 \cdot H(q^{(t)}, p^{(s)}),$$

where $H(\cdot, \cdot)$ is the cross-entropy. The factor T^2 compensates for the gradient scaling introduced by temperature smoothing.

The intuition is that *soft labels* encode richer information about class similarities than hard labels, for example, a teacher might assign $[0.6, 0.3, 0.1]$ to three classes, revealing relationships that a one-hot label like $[1, 0, 0]$ would hide. This richer signal allows the student to generalize better, even when it has fewer parameters.

In semi-supervised learning, the Mean Teacher framework[57] introduced the idea of updating the teacher network as an Exponential Moving Average (EMA) of the student’s weights and enforcing consistency between their predictions on differently augmented inputs. Unlike earlier approaches such as Temporal Ensembling, which averaged label predictions and updated them only once per epoch, Mean Teacher averaged the model weights directly, allows more frequent updates, improved scalability to large datasets, and better performance with fewer labels. Although originally developed for semi-supervised settings, its EMA teacher, student paradigm became a core component of many modern self-supervised methods.

Contrastive and Non-Contrastive Paradigms Contrastive learning methods such as Momentum Contrast (MoCo)[58] and SimCLR[59] established a strong foundation for SSL by training models to maximize agreement between different augmented views of the same image (positives) while pushing apart

views from different images (negatives).

In computer vision, contrastive methods can be interpreted as building a dictionary of encoded samples (“keys”) against which queries are compared: matching keys should be similar, while others should be dissimilar. For effective learning, such a dictionary should ideally be first both *large*, to adequately sample the continuous, high-dimensional visual space, and *consistent*, so that keys are represented by the same or similar encoder over time, ensuring stable comparisons during training.

Momentum Contrast (MoCo)[58] addressed these requirements by maintaining the dictionary as a queue of encoded samples, where new keys from the current mini-batch are enqueued and the oldest are dequeued, decoupling dictionary size from mini-batch size and allowing it to be large. To preserve consistency, MoCo reused the Exponential Moving Average (EMA) update idea from Mean Teacher[57], replacing the teacher with a *momentum encoder* that updates slowly as a moving average of the query encoder’s parameters. In summary:

- **Queue-based dictionary:** solves the batch-size limitation and allows the dictionary to be large.
- **Momentum encoder (EMA):** solves the consistency problem.

While MoCo focused on overcoming batch-size limitations and ensuring feature consistency through a momentum encoder, SimCLR[59] took an orthogonal approach by discarding the memory bank entirely and instead relying on very large batch sizes to supply sufficient negatives within each training step. Beyond this simplification, SimCLR systematically analyzed its components and identified three key factors that drive high-quality representations:

- **Data augmentation composition:** carefully designed combinations of augmentations are critical for defining effective predictive tasks.
- **Non-linear projection head:** introducing a learnable projection layer between the encoder and the contrastive loss substantially boosts performance.
- **Scale of training:** larger batch sizes and longer training schedules provide significant gains over supervised baselines.

Although SimCLR demonstrated the potential of contrastive learning without a memory bank, it still relied on explicitly sampling negative pairs. Bootstrap Your Own Latent (BYOL)[60] removed this requirement entirely, showing that high-quality representations can be learned without negatives. BYOL employs two networks: an *online network* and a *target network*. The online network is trained to predict the representation produced by the target network for a differently augmented view of the same image. To maintain stability and prevent representational collapse, the target network is updated as a slow-moving Exponential Moving Average (EMA) of the online network’s weights, an idea conceptually similar to Mean Teacher[57]. Key components that are important for this thesis, later influenced DINO include:

- **EMA target network:** provides a stable reference for the online network during training.
- **Asymmetric architecture:** online network includes an additional predictor MLP, while the target network does not, helping avoid collapse.

- **View invariance without negatives:** learns by matching representations of different augmented views of the same image.

Another important contribution came from the paper *Swapping Assignments between Views (SwAV)*[61], which removed the need for explicit pairwise feature comparisons by jointly clustering image representations into a set of learnable *prototypes* and predicting the prototype assignments of one view from another. This design made SwAV more memory-efficient and scalable, as it did not require large memory banks, while still achieving competitive performance without a momentum encoder. SwAV also introduced the *multi-crop* augmentation strategy, combining high- and low-resolution views to improve performance without additional computational cost, a technique later adopted by DINO[39].

Masked Autoencoding and Hybrid Methods Masked image modeling (MIM) is a self-supervised learning paradigm in which a portion of the input image is masked, and the model is tasked with predicting the missing content from the visible context. By forcing the network to infer missing information, MIM encourages the capture of global semantic structure rather than relying solely on local patterns.

In parallel to distillation-based methods, Masked Autoencoders (MAE)[62] pioneered a simple yet effective approach: randomly masking a very high proportion (typically 75%) of image patches, encoding only the visible ones with a lightweight Vision Transformer (ViT) encoder, and reconstructing the missing patches via a small, asymmetric decoder. This design greatly reduced computation, as the encoder processed fewer tokens, while encouraging the model to capture global context for accurate reconstruction.

MAE achieved strong scalability and transfer performance, but its pixel-reconstruction objective tends to prioritize low-frequency color and texture information rather than high-level semantic features. This limitation arises because the loss is computed directly in pixel space, pushing the model to reproduce precise visual details instead of abstract object-level representations. Since DINO[39] was introduced before the emergence of masked image modeling approaches such as MAE, it did not incorporate masking into its training objective. Later, its masked modeling principle was incorporated into DINOv2[40], where it was combined with self-distillation and feature-level prediction. This integration retained the efficiency benefits of masking while shifting the learning signal toward semantically meaningful representations, thus mitigating the primary weakness of pure pixel-based reconstruction.

Self-Distillation and ViT-Based SSL: DINO Building on BYOL’s momentum-based teacher–student architecture and SwAV’s clustering-based representation learning, DINO (*Distillation with No Labels*)[39] (2021) adapted self-distillation to *Vision Transformers* (ViTs). DINO trains two networks of identical architecture — a *student* and a *teacher* — where the teacher’s parameters θ_t are updated as an Exponential Moving Average (EMA) of the student’s parameters θ_s :

$$\theta_t \leftarrow m \theta_t + (1 - m) \theta_s,$$

with momentum $m \in [0, 1)$ (typically starting around 0.996 and increased to 1.0 over training).

The student receives a set of *global* and *local* crops (multi-crop augmentation), while the teacher sees only the *global* crops. Both networks output a probability distribution over a fixed number of prototypes (learnable cluster centers) using a projection head $g(\cdot)$ and a softmax with temperature scaling.

Given a normalized feature vector $\mathbf{z}$, the output distribution is:

$$p(\mathbf{z}, T) = \frac{\exp(\mathbf{z}/T)}{\sum_{k=1}^K \exp(\mathbf{z}_k/T)},$$

where T is the softmax temperature, controlling distribution sharpness.

To stabilize training and avoid *collapse to a constant representation*, DINO applies:

- **Centering**: maintaining a running mean $\mathbf{c}$ of the teacher’s outputs and subtracting it before softmax:

$$p_t = \text{softmax} \left(\frac{\mathbf{z}_t - \mathbf{c}}{T_t} \right),$$

ensuring balanced use of all prototypes.

- **Sharpening**: using a lower temperature T_t (e.g., 0.04) for the teacher to produce peaky target distributions, which are easier for the student to match.

The training objective is a cross-entropy between the teacher’s and student’s output distributions over all pairs of views where the student crop differs from the teacher crop:

$$\mathcal{L} = \frac{1}{N_{\text{pairs}}} \sum_{(i,j)} H(p_t^{(i)}, p_s^{(j)}),$$

where $H(\cdot, \cdot)$ is the cross-entropy, $p_t^{(i)}$ is the teacher distribution for view i , and $p_s^{(j)}$ is the student distribution for view j .

DINO’s design integrates key ideas from prior SSL frameworks:

1. **Momentum-updated teacher** (BYOL[60]): stabilizes the target representation over time.
2. **Prototypes** (SwAV[61]): learned cluster assignments optimized via soft labels.
3. **Multi-crop training** (SwAV[61]): enforces view-invariance across scales.
4. **Centering and sharpening** (DINO[39]): prevent representational collapse and improve uniformity.

DINO was one of the first to demonstrate that *pure transformer-based architectures* without convolutional inductive biases can be trained from scratch in a fully self-supervised manner, producing linearly separable features that rival supervised pretraining on ImageNet.

DINOv2: Extending Self-Distillation with Masked Image Modeling *DINOv2*[40] (2023) extends the DINO framework by combining its EMA teacher-student setup and as mentioned prototype-based clustering with masked image modeling (MIM) inspired by MAE[62]. The goal is to leverage the semantic consistency of self-distillation while incorporating the robustness and scalability benefits of generative masked modeling.

DINOv2’s design inherits:

1. **Momentum-updated teacher** (BYOL[60], DINO[39]): same EMA update mechanism to stabilize target representations.

2. **Prototypes** (SwAV[61], DINO[39]): soft cluster assignments learned jointly with the backbone.
3. **Multi-crop training** (SwAV[61], DINO[39]): enforces scale-invariant representations.

The main innovation is the integration of MIM into the self-distillation objective. A high proportion of image patches are randomly masked, and the student is tasked with predicting the teacher’s latent representations of the masked regions rather than raw pixels. This addresses a key limitation of MAE, where the pixel-reconstruction objective overemphasizes low-level details, by aligning masked patch predictions with semantically meaningful features.

The total training loss combines the self-distillation loss from DINO with the masked modeling loss:

$$\mathcal{L} = \mathcal{L}_{\text{SSL}} + \lambda_{\text{MIM}} \mathcal{L}_{\text{MIM}},$$

where λ_{MIM} balances discriminative and generative learning. This joint objective encourages both global semantic alignment and robust local feature recovery.

DINOv2 scales this approach to billions of images from uncured datasets, incorporating architectural refinements (e.g., larger ViT backbones, improved positional embeddings) and optimization improvements (e.g., longer cosine schedules, stronger regularization). The resulting features demonstrate exceptional transfer performance across classification, detection, and segmentation tasks without task-specific fine-tuning.

3.2 Self-Supervised Learning in Agriculture

The challenges of acquiring large-scale labeled datasets are particularly acute in agriculture, where annotation requires expert knowledge and is both time-consuming and costly[63]. This has motivated increasing interest in label-efficient and self-supervised learning (SSL) approaches that can leverage abundant unlabeled imagery from UAVs, satellites, and field cameras.

Several studies have explored SSL, and particularly the DINO/DINOv2 family of methods, in agricultural contexts. Singh, Bidese, Dhakal, *et al.* [64] proposed a *Few-Shot Adaptation of Grounding DINO* for agricultural detection tasks such as weed identification, fruit counting, and insect classification. Rather than relying on the zero-shot text prompts of Grounding DINO, which often fail to discriminate visually similar agricultural objects, they removed the text encoder and introduced a small set of trainable prompt embeddings. This lightweight adaptation enabled high accuracy in scenarios with only a handful of labeled samples, in many cases surpassing fully fine-tuned YOLO baselines on domain-specific datasets. Their results highlight the adaptability of DINO-derived architectures to data-scarce agricultural detection tasks.

In the remote sensing domain, Wanyan, Seneviratne, Shen, *et al.* [65] introduced DINO-MC, a variant of the original DINO framework tailored for multi-sized local crops and temporal view pairs. This design is particularly suited to aerial and satellite imagery, which often lacks the large-scale variability of natural images. Pretrained on remote sensing datasets, DINO-MC achieved state-of-the-art results on downstream tasks such as BigEarthNet land-cover classification, EuroSAT crop classification, and OSTL-CD change detection, even when trained with only a fraction of the available labeled data. These results demonstrate the effectiveness of adapting DINO’s augmentation and sampling strategy to the statistical properties of agricultural imagery.

Building on the integration of spatial and temporal context, Yan, Gu, and Chen [66] proposed CropSTS, a remote sensing foundation model for cropland classification. CropSTS combines a DINO-based self-supervised pretraining stage with a decoupled spatiotemporal attention architecture designed to capture both seasonal phenology and field-level spatial structure in Sentinel-2 satellite time series. To further improve representation quality, the authors employed knowledge distillation from a larger teacher model into the DINO-based backbone. CropSTS set new state-of-the-art benchmarks on the PASTIS-R dataset for parcel segmentation and crop-type classification, illustrating the utility of SSL in low-label, multi-temporal agricultural monitoring.

At the plant scale, Bai, Zhang, Gao, *et al.* [67] developed DINOv2-FCS, a multi-task model for fruit leaf disease classification and severity prediction. In their architecture, a DINOv2-B backbone serves as the feature extractor for both tasks, augmented with a Class, Patch Feature Fusion Module (C-PFFM) to integrate fine-grained lesion details with class-level context, and an Explicit Feature Fusion Architecture (EFFA) to improve lesion segmentation via adaptive atrous spatial pyramid pooling. On five fruit leaf disease datasets, DINOv2-FCS achieved 99.67% accuracy in disease classification and 90.29% mIoU in lesion segmentation, substantially outperforming prior CNN-based methods and maintaining strong generalization to unseen fruit species.

Earlier work by Kar, Nagasubramanian, Elango, *et al.* [68] also demonstrated the benefits of SSL for agricultural tasks, showing that self-supervised pretraining improved the classification of crop pests under limited labeled data. In a broader survey, Li, Chen, Qi, *et al.* [63] reviewed label-efficient learning techniques in agriculture, including SSL, semi-supervised learning, and active learning, and identified the integration of SSL with transformer architectures as a promising research direction for overcoming annotation bottlenecks.

Collectively, these studies confirm that SSL, and in particular DINO/DINOv2 and their variants, can produce high-quality, transferable representations for a range of agricultural vision tasks, from leaf-level disease detection to field-scale crop mapping, with significantly reduced dependence on labeled data. However, despite these advances, to our knowledge, no prior work has conducted a controlled comparison between DINOv2, supervised ViTs, and CNNs for UAV-based crop disease detection under multiple label availability regimes. This thesis addresses this gap by systematically evaluating these architectures at 1%, 10%, and 100% labeled data availability, providing a comprehensive assessment of their strengths and weaknesses in both label-rich and label-scarce scenarios.

Collectively, these studies confirm that SSL, and in particular DINO/DINOv2 and their variants, can produce high-quality, transferable representations for a range of agricultural vision tasks, from leaf-level disease detection to field-scale crop mapping, with significantly reduced dependence on labeled data. However, despite these advances, to our knowledge, no prior work has conducted a controlled comparison between DINOv2, supervised ViTs, and CNNs for UAV-based crop disease detection under multiple label availability regimes. This thesis addresses this gap by systematically evaluating these architectures at 1%, 10%, and 100% labeled data availability, providing a comprehensive assessment of their strengths and weaknesses in both label-rich and label-scarce scenarios.

As summarized in Table 3.1, prior studies highlight the potential of SSL in agriculture, particularly with DINO/DINOv2-based approaches, but none replicate the controlled experimental setup of this thesis. Here, we use a UAV-captured agricultural imagery dataset selected for its high resolution and field-level disease annotation quality, making it an ideal benchmark for evaluating self-supervised

Table 3.1: Summary of Key Agricultural SSL Works Using DINO/DINOv2 or Derivatives

<i>Reference</i>	Model	Application Domain	Labeling Approach	Key Innovation
Singh, Bidese, Dhakal, <i>et al.</i> [64]	Grounding DINO	Weed detection, fruit counting, insect classification	Few-shot learning	Text encoder removal with prompt embeddings for agricultural few-shot tasks
Wanyan, Seneviratne, Shen, <i>et al.</i> [65]	DINO-MC	Remote sensing	Semi-supervised (10% labels)	Multi-scale crops and temporal pairing for aerial imagery SSL
Yan, Gu, and Chen [66]	CropSTS	Satellite time-series	Semi-supervised	Spatiotemporal attention decoupling with knowledge distillation
Bai, Zhang, Gao, <i>et al.</i> [67]	DINOv2-FCS	Plant disease detection	Fully supervised	Joint disease classification and severity segmentation
Kar, Nagasubramanian, Elango, <i>et al.</i> [68]	SSL-CNN	Crop pest detection	Semi-supervised	SSL effectiveness demonstration for pest identification

DINOv2, supervised Vision Transformers (ViTs), and supervised CNNs under both low-label and full-label regimes. The following Methodology section details the dataset composition, preprocessing steps, model configurations, and evaluation protocol used in this comparative analysis.

4 Research Questions / Objectives

Main Research Question: How does the paradigm of self-supervised learning (DINO) compare to traditional supervised method Convolutional Neural Network (CNN) in navigating the critical trade-offs between data scarcity and label scarcity for fine-grained agricultural image classification?

4.1 Accuracy & Generalization

Under extreme data scarcity constraints, does the intrinsic local feature bias of a Convolutional Neural Network (CNN) provide a definitive performance advantage over DINO architecture?

4.2 Label Efficiency

When total data volume is large but label availability is severely limited, how DINO compare to CNN?

4.3 Model Complexity & Efficiency

When comparing CNN and DINO models, which architecture is more resource efficient? Measuring the time required to reach peak performance (training convergence) and the speed of prediction (inference time) and how do these efficiency differences affect practical deployment on edge farming equipment or mobile devices?

5 Methodology

This chapter details the experimental methodology and setup used to compare a self-supervised Vision Transformer (DINOv2), built upon a ViT backbone, against a supervised convolutional neural network (CNN, EfficientNet-B4) under various data constraints. To disentangle backbone architecture from learning paradigm of DINOv2, a classic supervised Vision Transformer (ViT) is also included as a baseline, isolating gains due to self-supervised pretraining rather than the ViT backbone.

For DINOv2, two fine-tuning regimes were evaluated: in the linear probe (LP) regime, the backbone is frozen and only a lightweight classification head is trained, measuring the direct transferability of the self-supervised features without adaptation; in the full fine-tune (FT) regime, both the backbone and classification head are trained using progressive unfreezing, allowing the model to adapt its self-supervised features to the target agricultural domain. The study systematically analyzes model performance across a spectrum of training resource availability, including varying label fractions (1%, 10%, 50%, and 100%) and varying data fractions (5%, 25%, and 50%) to simulate small farm environments.

The methodology also incorporates specific DINOv2 hyperparameter variations and rigorous testing of dynamic quantization, providing critical insights for optimizing these models for efficient edge deployment in agricultural image classification.

5.1 Dataset

Numerous datasets have been developed for various agricultural computer vision tasks, spanning a wide spectrum of applications from crop classification to semantic segmentation and remote sensing.

5.1.1 Noteworthy Datasets

- **Rice Image Dataset**
 - **Data Points:** 75,000 high-resolution images evenly distributed across five distinct rice varieties.
 - **Applications:** Used for grain variety classification, quality assessment, and automated sorting.
- **Space2Ground Dataset**
 - **Data Points:** A multi-modal collection that integrates Sentinel-1/2 satellite data with corresponding ground-level Mapillary photographs.
 - **Applications:** Relevant for land-use classification, grassland monitoring, and remote sensing validation.
- **Canadian Cropland Dataset**
 - **Data Points:** 78,536 geo-referenced image patches, featuring data from 12 spectral bands and various vegetation indices (NDVI, GNDVI).
 - **Applications:** Applicable to crop classification, multi-temporal analysis, and general land cover monitoring.

- **NAIP (National Agriculture Imagery Program)**

- **Data Points:** High-resolution (1m GSD) aerial imagery covering the United States, with data available from 2003 to the present, and includes RGB and Near-Infrared (NIR) bands.
- **Applications:** Used widely in remote sensing, land classification, and long-term agricultural monitoring.

- **CoFly-WeedDB**

- **Data Points:** A focused set of 201 RGB images captured via UAV over a cotton field in Larissa, Greece, and annotated for three distinct weed types: Johnson grass, Field bindweed, and Purslane.
- **Applications:** Designed for automatic weed segmentation and classification, supporting AI-based methodologies for precision agriculture.

5.1.2 Chosen Dataset

Dataset Overview

The dataset, sourced from Kaggle, contains 24,881 labeled images from farms in Ghana, covering four major crops and 22 disease/pest categories. Images were collected under uncontrolled field conditions, characterized by:

- High intra-class variability
- Diverse environmental conditions (lighting, background, occlusion)
- Real-world field noise

This dataset was used to benchmark classification performance and evaluate trade-offs between accuracy and efficiency.



Figure 5.1: Example from the Crop Pest and Disease Detection Dataset.

5.1.3 Rationale for Dataset Selection

The Crop Pest and Disease Detection Dataset from Ghana was selected over other agricultural datasets, including the CoFly-WeedDB, due to its direct alignment with the specific experimental goals of this study: benchmarking classification robustness and efficiency under resource constraints.

- **Task Alignment and Domain Complexity:** This study focuses on multi-class classification across multiple crops and disease/pest categories. The Ghana dataset, with 22 classes and images collected under real-world field conditions, provides a diverse and complex environment for evaluating model performance. Other datasets, such as CoFly-WeedDB or the Rice Image Dataset, focus on fewer classes or more specialized tasks, which are less suited for testing the scalability of classification methods under varying data availability.
- **Scale and Resource Considerations:** With 24,881 images, the Ghana dataset allows robust experimentation across varying data (5%, 25%, 50%) and label fractions (1%, 10%, 50%, 100%), enabling a comprehensive analysis of data efficiency. Smaller datasets, such as CoFly-WeedDB (201 images), would require additional augmentation or sampling strategies to support these regimes. Larger datasets like NAIP or Space2Ground, while rich in data, would demand greater computational resources for preprocessing and training, increasing the experimental budget and limiting the feasibility of multiple fine-tuning scenarios.
- **Practical Suitability for the Study:** The Ghana dataset strikes a balance between task complexity, dataset size, and computational requirements, making it ideal for evaluating the performance of both self-supervised and supervised models under realistic small farm scenarios. Its characteristics also support testing linear probing, full fine-tuning, and efficient edge deployment, directly addressing the study’s experimental objectives.

In summary, the Ghana dataset was chosen for its appropriate scale, diverse classification complexity, and practicality for systematic evaluation under resource-constrained conditions, making it the most suitable benchmark for this study.

5.1.4 Why Classification Task

The entire methodology centers on the classification task for the following reasons, primarily serving to isolate and rigorously test model feature quality and deployment readiness:

- **Direct Measurement of Feature Quality:** The multi-class classification task serves as the purest and most direct benchmark for evaluating the quality of the learned representations. Success requires the model to distill complex visual data into a compact, highly discriminative feature vector, directly testing the effectiveness of DINOv2’s self-supervised pretraining.
- **Enabling Linear Probing (LP):** One of the tests of DINOv2’s representation quality is the Linear Probing experiment, which only trains the final linear layer. This regime is intrinsically linked to and only meaningfully evaluated through a classification objective.
- **Relevance to Edge Deployment Efficiency:** Classification is the most common and computationally lightest task deployed on resource-constrained edge devices for rapid initial assessment.

Focusing on classification ensures that the derived results for Training Time, Inference Speed, and Quantization directly address the practical limitations and goals of efficient edge computing.

5.2 Data Preparation

The data preparation pipeline was meticulously designed to facilitate effective fine-tuning of pre-trained models on agricultural imagery while rigorously simulating various resource constraints.

- **Dataset Partitioning:**

The full dataset was initially partitioned into three main sets: 70% for training, 15% for validation, and 15% for testing, using stratified sampling to preserve original class proportions across all splits. A fixed random seed (`RANDOM_STATE=42`) ensured the reproducibility of this initial split. Critically, the test set was generated once and kept fixed throughout all experiments, ensuring an unbiased final evaluation.

- **Label Efficiency Regimes (Label Hiding):**

To simulate conditions of limited labeling budgets, a *label hiding* approach was applied to the main 70% training set. For the few-shot regimes (10% and 50% labels), only the corresponding percentage of training labels were revealed, while the remaining labels were masked as -1.

- **Extreme Low-Label Case:**

A specific 1% label fraction was utilized to test the models' performance in an *extreme few-shot environment*. This regime directly evaluated the representational quality of the self-supervised models when exposed to minimal task-specific supervision. Also again that is done by masking the remaining labels as -1.

- **Data Efficiency Regimes (Data Fraction):**

To simulate environments with limited overall training data (i.e., small farms that cannot acquire 24,000 images), data fraction experiments were conducted using subsamples of the full training set at 5%, 25%, and 50% of the original training data size. This allowed for the comparative analysis of model stability and convergence under true data scarcity, where the input image volume itself is reduced.

- **Resizing:**

Images were resized to 224×224 pixels in order to keep the same input size for all models, therefore the models are evaluated on the same image size conditions.

- **Normalization:** Pixel values were normalized using the pretraining statistics of each model:

- mean $[0.485, 0.456, 0.406]$, std $[0.229, 0.224, 0.225]$

The reason is that because they are pre-trained models.

- **Augmentation (Training Set Only):** To improve robustness and generalization, different random augmentation policies were applied to the training sets of each model, tailored to the specific architecture. No augmentations were applied to the validation or test sets (all were evaluated at 224×224 resolution).

– **CNN (Strongest Augmentation):**

- * Random Affine (translation 0.2, rotation $\pm 20^\circ$)
- * Horizontal Flip (50% probability)
- * Vertical Flip (30% probability)
- * Color Jitter (up to 0.4 for brightness/contrast, 0.2 for saturation/hue, plus extra brightness/contrast adjustment)

– **ViT (Moderate Augmentation):**

- * Random Affine (translation 0.1, rotation $\pm 15^\circ$)
- * Horizontal Flip (50% probability)
- * Vertical Flip (20% probability)
- * Color Jitter (around 0.3/0.2/0.1 for different parameters)

– **DINOv2 (Lighter Augmentation):**

- * Random Affine (translation 0.1, rotation $\pm 10^\circ$)
- * Horizontal Flip (50% probability)
- * Vertical Flip (20% probability)
- * Color Jitter (brightness, contrast, saturation, hue = 0.2)

Rationale for Policy Differences The variation in augmentation strength was selected based on the architectural properties of each model:

- **CNN (Strongest):** CNNs are generally more susceptible to translational and rotational variance. The strong policy ($\pm 20^\circ$ rotation, 0.8–1.2 scale) was used to explicitly inject high diversity, forcing the model to learn more robust, position-invariant features and mitigating overfitting.
- **ViT (Moderate):** Vision Transformers, due to their self-attention mechanism, are inherently more robust to moderate geometric changes. A moderate policy ($\pm 15^\circ$ rotation, 0.9–1.1 scale) was used, as excessive augmentation can sometimes interfere with the global relational learning of the attention heads.
- **DINOv2 (Lighter):** As a model pre-trained via Self-Supervised Learning (SSL) on massive datasets, DINOv2 already possesses strong feature invariance. A lighter policy ($\pm 10^\circ$ rotation, minimal translation/color jitter) was applied during fine-tuning. This prevents overly aggressive modification of the already high-quality, generalized representations learned during pretraining, allowing for efficient adaptation to the downstream task.

Test Set: The test set was processed using only deterministic transformations to ensure a fair and reproducible evaluation:

- Resizing: Each image was resized to what was used in the training set (224×224).
- Normalization: Pixel values were normalized using the same statistics applied during model training.
- No random augmentations (e.g., flips, rotations, color jitter) were applied.

This ensured fair and reproducible evaluation, aligned with standard deep learning practices.

5.3 Model Architectures

This thesis leverages two distinct classes of powerful pre-trained deep learning models: a Convolutional Neural Network (CNN) from the EfficientNet family and a Vision Transformer (ViT) from the DINOv2 family. Both models were fine-tuned on the agricultural dataset. A common fine-tuning strategy, known as progressive unfreezing, was applied to both architectures to optimize performance while mitigating overfitting, especially given the smaller scale of the target dataset compared to the massive pretraining corpora.

5.3.1 Progressive Unfreezing Strategy

Progressive unfreezing is a widely adopted transfer learning technique for adapting pre-trained models to new datasets. It proceeds in two main stages:

1. **Initial Freezing:** In the early epochs of fine-tuning, the majority of the pre-trained model's layers (the backbone or feature extractor) were kept frozen. This means their weights were not updated during backpropagation. Simultaneously, the newly added classification head, which was randomly initialized, was trained to map the fixed features extracted by the backbone to the specific classes of the target dataset. This phase allowed the randomly initialized head to quickly learn to classify the new data using the robust, general-purpose features already learned by the pre-trained backbone. It was particularly effective at preventing the newly initialized layers from corrupting the valuable pre-trained weights of the backbone, which are often fragile to large gradients at the beginning of fine-tuning.
2. **Subsequent Unfreezing:** After a predetermined number of initial epochs (e.g., 3 epochs in our setup), the frozen backbone layers were unfrozen, making them trainable. This allowed the entire model to adapt its deeper, more general features to become more specialized for the nuances of the agricultural dataset. When unfreezing, training was continued with a lower learning rate for these previously frozen layers to ensure gradual adaptation and prevent *catastrophic forgetting* of the knowledge acquired during pretraining. This two-stage approach provided a stable initial convergence before allowing the full model capacity to be utilized for domain-specific fine-tuning, striking a balance between leveraging pre-trained knowledge and adapting to new data.

5.3.2 EfficientNet-B4 (Supervised CNN)

EfficientNet-B4 is a convolutional neural network architecture known for its strong performance across a wide range of computer vision tasks.

- **pretraining:** The model was pre-trained on the ImageNet-1k dataset, which consists of over a million images across 1,000 object categories. This extensive pretraining enabled EfficientNet-B4 to learn a rich hierarchy of visual features.
- **Architecture:** Key architectural innovations include *compound scaling*, which uniformly scales network depth, width, and resolution using a fixed set of scaling coefficients; *Mobile Inverted Bottleneck Convolutions (MBConv)* that form the building blocks of the network; and integrated *Squeeze-and-Excitation (SE) blocks*, which allow the model to dynamically recalibrate channel-wise feature responses.

Fine-tuning Strategy: For EfficientNet-B4, the progressive unfreezing strategy described above was employed. Specifically, the backbone layers (feature extractor) were initially frozen for a set number of epochs. After this initial phase, the entire model, including the backbone, was unfrozen and fine-tuned on the agricultural dataset, allowing for adaptation of both shallow and deep features.

5.3.3 DINOv2 (Self-supervised)

DINOv2 is a Vision Transformer (ViT) model distinguished by its self-supervised pretraining approach. Unlike CNNs, Vision Transformers process images as sequences of patches, enabling them to capture long-range dependencies and contextual relationships.

- **pretraining:** DINOv2 models were pre-trained in a fully self-supervised manner on a curated dataset of approximately 142 million unlabeled images (LVD-142M). This dataset was assembled via an automated pipeline from both established collections (e.g., ImageNet-22k) and web-sourced images, with processes for deduplication and quality control ensuring diverse and robust visual representations[40].

Fine-tuning Strategy: Similar to EfficientNet-B4, the pre-trained backbone of DINOv2 was initially frozen, allowing the newly attached classification head to adapt. Subsequently, the entire transformer backbone was unfrozen and fine-tuned, enabling the model to adjust its powerful self-supervised features to the specific characteristics of the agricultural imagery.

5.3.4 Vision Transformer (ViT, Supervised Baseline)

Vision Transformers (ViTs) process images as sequences of fixed-size patches, enabling global self-attention across spatial regions. This model shares the same backbone architecture as DINOv2 (e.g., `vit_base_patch16_224`), allowing a controlled comparison between supervised and self-supervised pretraining approaches.

- **pretraining:** The supervised ViT model was pre-trained on ImageNet-1k, leveraging labeled supervision to learn feature representations across 1,000 object classes.
- **Architecture:** The model consists of a patch embedding layer, a stack of transformer encoder blocks with multi-head self-attention, and a class token representing global context for classification.

Fine-tuning Strategy: Progressive unfreezing, identical to the approach described earlier, was employed. The backbone was frozen for the initial epochs to stabilize training of the newly initialized classification head, then gradually unfrozen with a reduced learning rate.

Purpose: This model serves as a supervised ViT control, enabling direct comparison with DINOv2 to isolate the contribution of self-supervised pretraining from architectural effects.

5.4 Training Framework

The training framework was designed to facilitate efficient and stable fine-tuning of pre-trained models on the agricultural dataset. This section details the training regimes, optimization strategies, and progressive fine-tuning approach employed.

5.4.1 Training Regimes and Data Availability Scenarios

Model performance was evaluated under different supervision and data availability scenarios. Experiments were conducted across four models:

- CNN (EfficientNet-B4, supervised)
- ViT (Supervised)
- DINOv2 (Self-Supervised, Linear Probe)
- DINOv2 (Self-Supervised, Fine-Tune)

Two types of experimental variations were considered:

Label Fraction Variation

In these experiments, the full training set was used, but only a fraction of the labels were visible to the model; the remaining labels were masked. This isolates the effect of label scarcity while keeping the input data constant.

- **Label fractions evaluated:**
 - 1% — Extreme low-label scenario
 - 10% — Few-shot, partial supervision
 - 50% — Moderate supervision
 - 100% — Full supervision
- **Models:**
 - DINOv2: Linear Probe (LP) and Fine-Tune (FT)
 - CNN and ViT: Progressive unfreezing applied

Data Fraction Variation

In these experiments, all labels were provided, but only a subset of the training images was used. This isolates the effect of dataset size on model performance.

- **Data fractions evaluated:**
 - 5% — Very small subset
 - 25% — Small dataset
 - 50% — Medium dataset
 - 100% — Full dataset
- **Models:** All four models were fine-tuned under these conditions.

Summary

This experimental design allows us to disentangle three factors:

1. **Architecture effects:** CNN vs ViT backbones
2. **Pretraining strategy effects:** Supervised vs Self-Supervised
3. **Data availability effects:** Label scarcity vs Dataset size

By separating label fraction and data fraction experiments, each set isolates a single variable, ensuring clear interpretation of the results.

5.4.2 Optimization

The optimization process was critical for efficient and effective model training. Due to architectural differences and pretraining paradigms, model-specific hyperparameters were employed:

- **Optimizer:** AdamW (Adaptive Moment Estimation with Weight Decay) was used across all models, as it decouples weight decay from gradient-based updates, improving generalization. However, the momentum parameters differed by architecture:
 - **CNN and ViT:** $\beta_1 = 0.9, \beta_2 = 0.999$ (standard settings for supervised training)
 - **DINOv2:** $\beta_1 = 0.9, \beta_2 = 0.95$ (optimized for self-supervised models)
- **Learning Rate and Warmup:**
 - **CNN (Supervised):** Initial LR 1×10^{-3} with 3-epoch linear warmup from 1×10^{-6}
 - **ViT (Supervised):** Initial LR 5×10^{-4} with 3-epoch linear warmup from 1×10^{-6}
 - **DINOv2 (Linear Probe):** LR $\leq 2 \times 10^{-3}$, no warmup (frozen backbone requires less aggressive optimization)
 - **DINOv2 (Fine-Tune):** LR capped at 1×10^{-4} with 10-epoch linear warmup from 1×10^{-7} (extended warmup to preserve self-supervised representations)
- **Learning Rate Scheduling:**
 - **CNN:** ReduceLROnPlateau with patience=5, factor=0.5
 - **ViT (Supervised):** Cosine annealing for smooth decay over training
 - **DINOv2 (Linear Probe):** Step scheduler (step size=7, gamma=0.5)
 - **DINOv2 (Fine-Tune):** Cosine annealing with extended warmup
- **Regularization:**
 - **Weight Decay:** 1×10^{-2} for CNN and ViT; 5×10^{-2} for DINOv2 fine-tuning (stronger regularization for larger pre-trained models)
 - **Dropout:** 0.1 for DINOv2, 0.15 for ViT, 0.2 for CNN (varied to match model capacity and overfitting risk)

- **Training Duration:**

- CNN and ViT (Supervised): 75 epochs
- DINOv2 (Linear Probe): 60 epochs
- DINOv2 (Fine-Tune): 75 epochs

- **Early Stopping:** Training was terminated if the validation loss did not improve for a patience of 3 epochs (default) or 10 epochs (for few-shot scenarios), preventing overfitting and optimizing computational resources.

5.4.3 Progressive Unfreezing and Hyperparameter Variations

Progressive unfreezing[69] was employed to adapt pre-trained models to the agricultural dataset. This strategy mitigates overfitting for smaller datasets while leveraging pre-trained knowledge.

General Strategy

1. **Initial Freezing:** The backbone (feature extractor) was frozen for an initial period; only the classification head was trained. This allows the head to adapt using stable features without destabilizing pre-trained weights.
2. **Unfreezing:** After the initial frozen period, the backbone was unfrozen. The learning rate for all parameters was reduced by a factor of 10 (`UNFREEZE_LR_FACTOR=0.1`) to prevent catastrophic forgetting. This gradual adaptation allows the model to specialize representations for the agricultural task.

Model-Specific Configurations

- **CNN (EfficientNet-B4, Supervised):**

- Unfreeze epoch: 3
- Warmup duration: 3 epochs
- Rationale: CNNs rely on hierarchical convolutional features, which are moderately sensitive to small-scale domain shifts. Early unfreezing allows the convolutional layers to adapt efficiently to new, task-specific patterns in the agricultural dataset without destabilizing the pre-trained weights.

- **ViT (Supervised):**

- Unfreeze epoch: 3
- Warmup duration: 3 epochs
- Rationale: ViTs encode global relationships via patch-based self-attention. Early unfreezing allows the attention layers to specialize their representations to the target domain, while the short warmup prevents large gradient updates from disrupting pre-trained relational features.

- **DINOv2 (Linear Probe):**

- Backbone remains frozen for the entire training

- Train only the classification head for 60 epochs
- Rationale: Evaluates transferability of self-supervised features without adaptation.
- **DINOv2 (Fine-Tune):**
 - Unfreeze epoch: 15
 - Warmup duration: 10 epochs
 - Warmup schedule: Linear ($0.000100 \rightarrow 0.001$)
 - Post-unfreeze LR: 1×10^{-4}
 - Rationale: Self-supervised pretraining on 142M images produces robust features. Longer frozen period and extended warmup preserve these features and avoid catastrophic forgetting.
- **DINOv2 (Fine-Tune, Hyperparameter Variation):**
 - Unfreeze epoch: 3
 - Warmup duration: 3 epochs
 - Warmup schedule: Linear ($0.000333 \rightarrow 0.001$)
 - Post-unfreeze LR: 1×10^{-4}
 - Rationale: A small variation in unfreeze timing, warmup duration, and initial learning rate was tested to study the sensitivity of model adaptation to hyperparameter choices. Results are reported later in chapter 6 Results and Discussion.

Summary This approach ensures that:

1. Supervised model (CNN) can adapt aggressively due to moderate pretraining dataset sizes.
2. Supervised model (ViT) can maintain same backbone to differentiate learning paradigms.
3. Self-supervised DINOv2 features are fine-tuned conservatively to preserve generalization.
4. Hyperparameter sensitivity is explicitly studied through a small variation experiment, highlighting the impact of unfreeze timing and learning rate schedules on model performance.

5.5 Evaluation Protocol

The evaluation protocol was designed to rigorously assess model performance in terms of classification accuracy and operational efficiency, with particular emphasis on deployment feasibility for resource-constrained agricultural drones.

5.5.1 Metrics

Model performance was assessed using a combination of classification and efficiency metrics:

- **Classification Metrics:**
 - **Accuracy:** Overall proportion of correctly classified test samples.

- **Precision, Recall, and F1-score:** Computed per-class and aggregated using both macro-averaging (unweighted) and weighted averaging (by class support) to evaluate performance across imbalanced classes.
- **Matthews Correlation Coefficient (MCC):** A robust single-value metric ranging from -1 to +1 that accounts for class imbalance and all four confusion matrix categories. Higher values indicate better overall classification quality.
- **Efficiency Metrics:** To assess deployment suitability on embedded systems:
 - **Model Size (MB):** Disk size of saved model weights.
 - **Size Reduction (%):** Percentage decrease in model size after quantization.
 - **Inference Latency (ms/image):** Average per-image inference time measured on CPU (for quantized models) and GPU (for original models).
 - **Speed Improvement (%):** Percentage decrease in inference time after quantization.
 - **Accuracy Drop (%):** Difference in test accuracy between original and quantized models.

All efficiency metrics were measured for both original (FP32/FP16) and dynamically quantized (INT8) models.

5.5.2 Quantization and Deployment Optimization

To optimize models for edge deployment, post-training dynamic quantization was applied:

- **Mixed-Precision Training (FP16):** Automatic Mixed Precision (AMP) was employed during GPU-based training and inference, using FP16 for compatible operations while retaining FP32 where necessary. This reduced GPU memory usage and accelerated training with minimal accuracy loss.
- **Dynamic Quantization (INT8):** Post-training dynamic quantization was applied to all models using PyTorch’s `fbgemm` backend. For CNNs (EfficientNet-B4) and supervised ViT, both `nn.Linear` and `nn.Conv2d` layers were quantized to INT8. For DINOv2 (transformer-based), only `nn.Linear` layers were quantized due to architectural constraints. This technique converts weights to INT8 and dynamically quantizes activations at inference time, significantly reducing model size (typically 50-75% reduction) and improving CPU inference speed, with minimal accuracy degradation (typically $\leq 1-2\%$).
- **Evaluation Environment:** Quantized models were benchmarked on CPU (Intel architecture with `fbgemm` support), reflecting typical edge deployment scenarios. Original models were evaluated on GPU (CUDA) during training and testing. This dual-platform approach ensures that efficiency gains are measured under realistic deployment conditions: GPU for development and training, CPU for on-device inference on agricultural drones.
- **Static PTQ and QAT:** Static post-training quantization and quantization-aware training were considered but not implemented. Static PTQ requires representative calibration data and additional profiling; QAT requires retraining with quantization simulation. Dynamic quantization was chosen for its ease of deployment and compatibility across architectures, providing a practical balance between accuracy retention and efficiency gains.

This protocol ensures comprehensive evaluation of both predictive performance and deployment viability, aligning the study with real-world agricultural drone constraints.

5.6 Reproducibility

Ensuring reproducibility is critical for scientific rigor. The following measures were implemented to maximize reproducibility, logging, and performance.

5.6.1 Deterministic Training

- **Fixed Global Seeds:** All random number generators were seeded with a fixed value (42):

- `random.seed(42)`
- `numpy.random.seed(42)`
- `torch.manual_seed(42)`
- `torch.cuda.manual_seed(42)`

This ensures consistent weight initialization, data shuffling, and augmentation patterns across runs.

5.6.2 Logging and Checkpointing

- **Training Logs:** Epoch-wise metrics (loss, accuracy, learning rate, overfitting warnings) via `TrainingLogger`.
- **Evaluation Logs:** Per-class precision, recall, F1-score, confusion matrices via `EvaluationLogger`.
- **Model Checkpoints:** Best model (by validation accuracy) and periodic checkpoints saved with:

- `model.state_dict`
- `optimizer.state_dict`
- Epoch number, validation performance
- Hyperparameters and training configuration

The following subsections describe the hardware and software environment, and the results of the experiments.

5.7 Experimental Setup

All experiments were conducted on a consistent hardware and software platform to ensure fair and reproducible comparisons. The optimized configuration used for training and evaluation is summarized below:

- **Hardware:** NVIDIA A100 80GB GPU.
- **Batch Size:** 640 per GPU step.
- **Data Loading:** 20 parallel workers with persistent workers and `pin_memory` enabled.

- **Mixed Precision:** Automatic Mixed Precision (AMP) with FP16/FP32 to reduce memory usage and accelerate training.
- **Gradient Accumulation:** To enable training on GPUs with smaller memory in case of time or budget constraints, gradient accumulation was implemented; for example, a physical batch size of 128 with five accumulation steps achieves the same effective batch size of 640, albeit with proportionally longer training time. This was not necessary for the current setup.
- **Software:** Python 3.12 was used, with a virtual environment (venv) to isolate dependencies and ensure reproducibility. The Rust-based package manager `uv` was employed for faster environment management compared to `pip`. Key libraries included `torch`, `torchvision`, `timm`, `scikit-learn`, `numpy`, `matplotlib`, `seaborn`, `tqdm`, and `Pillow`.

This configuration ensured efficient and reproducible training and evaluation of all models while providing flexibility to adapt to smaller GPU resources if needed.

6 Results and Discussion

This chapter presents and interprets the empirical findings derived from the experimental framework described chapter 5 Methodology. The results are organized to reflect the key research dimensions explored in this study:

The influence of data and label availability on model performance, the sensitivity of DINOv2 to fine-tuning hyperparameters, and the efficiency of quantized models for deployment on resource-constrained hardware.

Each section discusses both quantitative outcomes and qualitative observations, connecting them to the broader goals of evaluating self-supervised and supervised models under agricultural imaging conditions. The chapter concludes with a comparative discussion that synthesizes these insights and highlights the trade-offs between accuracy, efficiency, and adaptability

6.1 Results Across Data and Label Fractions

6.1.1 Data Scarcity

We study performance when limiting the number of fully labeled training images to fractions of the full dataset (e.g., 5%, 25%, 50%). Top-1 accuracy and macro-F1 results are reported.

The data fraction experiments were designed to assess how model performance scales when the total amount of available imagery is restricted. In otherwords, when images are limited, such as a very small farm. Under the most constrained regime (5% of the data), it was observed that a CNN baseline achieved stronger performance than ViT (CNN achieved 45.7% accuracy and ViT 20.7.% accuracy), whereas DINO-based approaches were competitive and superior (DINO FT 57.4% accuracy, DINO LP 56.9% accuracy). This behavior suggests that the inductive biases and locality captured by convolutional filters can offer robustness when very few samples are available, while the self-supervised DINO representations confer advantages even with limited fine-tuning data. The comparatively weaker performance of ViT at 5% indicates that supervised transformer-based architectures may require either more data or stronger regularization to stabilize optimization in the small-data regime.

As the data fraction was increased to 25%, CNN got the lead (from 45.7% to 90%) and ViT approached to the levels of DINO FT and DINO LP (from 20.7% to 85.4% respectively). DINO converged in performance (DINO FT from being the leader with 57.4% to 83.7% and DINO LP from 54.9% to 84%) and remained competitive with the CNN. This pattern indicates that once a modest amount of data is available, representation capacity decreases, narrowing gaps between architectures. Especially for this agricultural dataset, where the images are not so different than ImageNet. At 50% data, the CNN remained the top performer, although the accuracy has almost not changed, with ViT second and DINO third, especially DINO LP performed quite competitive. This ranking implies that, in this dataset and training setup, the CNN inductive bias continues to translate into strong generalization as data increases, while DINO’s advantage from pretraining and representation quality is more pronounced at the lowest data levels but does not necessarily dominate as more data and labels are provided.

Overall, these results indicate that data scarcity impacts architectures differently. Convolutional inductive biases appear particularly beneficial at moderate data fractions, ViT benefits more once a

threshold of data is reached, and DINO exhibits strong resilience. At very low data SSL method of DINO, shows clearly the power of not needing too many data to perform well. Albeit with diminishing relative advantage as more data and labels are provided.

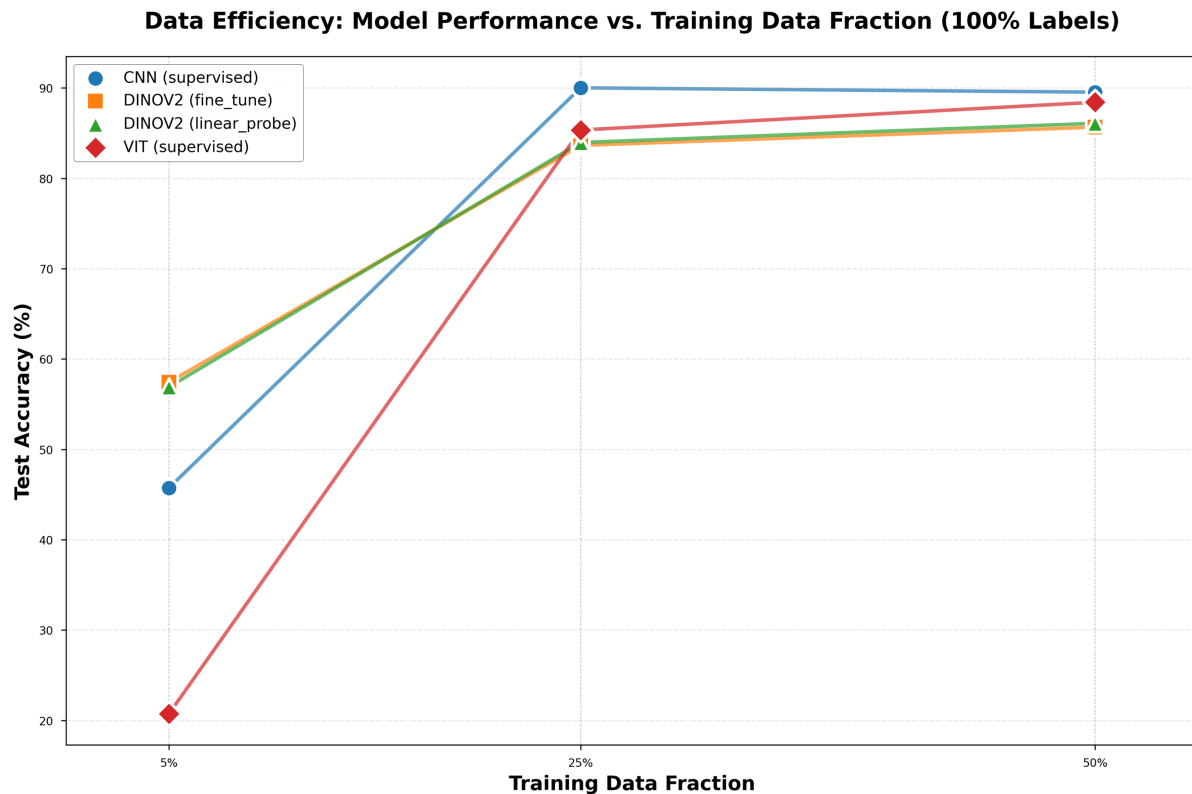


Figure 6.1: Data fraction comparison.

The quick overview in Figure 6.1 highlights how sensitivity to data sample scarcity differs across methods. At 5% data there is a pronounced separation: DINO variants clearly lead, demonstrating that self-supervised pretraining yields representations that are robust with only a handful of labeled examples. Supervised ViT performs worst in this regime, while the CNN baseline benefits from strong inductive locality priors. As the data fraction grows, the curves converge, indicating that additional labeled data reduces estimator variance and enables architectures with less pretraining to close the gap. By 25% and 50% the CNN frequently matches or surpasses DINO, showing that the relative advantage of SSL representations diminishes as more labeled data becomes available and simple inductive biases continue to provide reliable generalization.

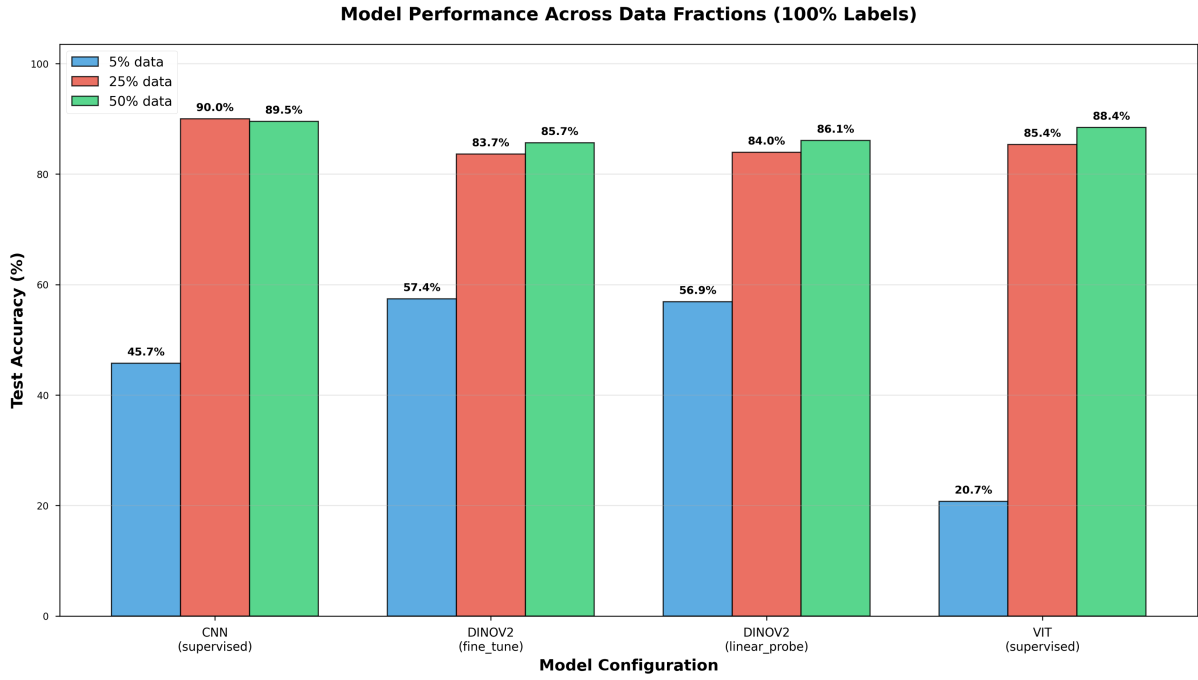


Figure 6.2: Data fraction comparison.

The accuracy plot in Figure 6.2 reveals three regimes. In the quite low-data regime (5% of the total images, 1,200 images), self-supervised DINO variants lead (DINO FT 57.4%, DINO LP 56.9%), substantially outperforming both the CNN baseline (45.7%) and supervised ViT (20.7%), showing that pre-trained, representation-rich features are uniquely valuable when very few training images are available. In the mid-data regime (25% of the total images, 6,000 images), the gap narrows and the CNN displays a sharp jump in accuracy (90%). This surge likely reflects the alignment between convolutional inductive biases and the relatively structured, texture-rich nature of agricultural imagery: local filters readily capture disease patterns such as spots and discolorations. Additionally, CNNs tend to be more stable under limited hyperparameter tuning and data augmentation, while transformers often require more precise optimization and regularization to reach similar stability, which I will show in DINO, SSL case especially in the Hyperparameter Sensitivity subsection 6.1.4. ViT (85.4%) and the DINO variants (DINO FT 83.7%, DINO LP 84%) converge in this regime as increased image data allows models to exploit their representation capacity more fully. By 50% of the total images (12,000), performances cluster more tightly, the CNN retains the lead (89.5%), with ViT (88.4%) and DINO variants (DINO FT 85.7%, DINO LP 86.1%) close behind, demonstrating that the CNN’s inductive biases continue to provide robust generalization as more images are available.

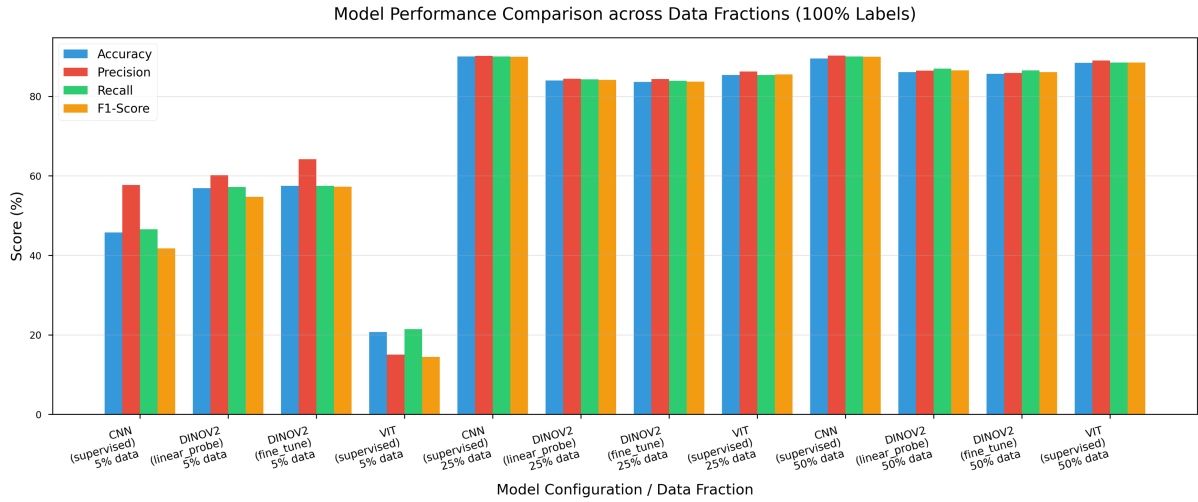


Figure 6.3: Data fraction comparison.

The Macro-F1 score (orange bars in Figure 6.3), as the unweighted average of per-class F1 scores, complements accuracy by providing an equal assessment of performance across all classes, irrespective of their frequency. The increase in Macro-F1 from 5% to 25% data suggests improved representation for minority classes. The CNN model achieves the highest overall performance at the 50% data fraction, reflecting broad gains distributed across classes, while the relatively smaller improvement for DINOv2 at 50% suggests its strongest benefit is realized in the most data-constrained settings, where its pre-trained features disproportionately aid minority classes. As we have mentioned before.

Detailed Experiment Results (100% Labels, Variable Data)

Model	Regime	Budget	Train Acc	Test Acc	Time (s)
CNN	supervised	5% data	59.18%	45.74%	38.9
DINOv2_FT	fine_tune	5% data	63.09%	57.45%	53.2
DINOv2_LP	linear_probe	5% data	56.83%	56.91%	28.3
ViT	supervised	5% data	21.99%	20.74%	22.3
CNN	supervised	25% data	96.22%	90.02%	193.8
DINOv2_FT	fine_tune	25% data	23.22%	83.65%	102.4
DINOv2_LP	linear_probe	25% data	87.65%	83.97%	281.3
ViT	supervised	25% data	94.04%	85.35%	102.1
CNN	supervised	50% data	97.44%	89.54%	419.8
DINOv2_FT	fine_tune	50% data	71.41%	85.67%	231.1
DINOv2_LP	linear_probe	50% data	88.22%	86.09%	389.4
ViT	supervised	50% data	94.68%	88.43%	192.4

Figure 6.4: Data fraction comparison. Time is in seconds. They have finished relatively quickly because of A100 80GB GPU. But more importantly models have not finished their intended training time, which we will discuss later in section 6.3 Comparative Discussion.

6.1.2 Label Scarcity

Here the number of images fixed and vary the fraction of images with labels (semi-supervised setup). This isolates the effect of label availability from dataset size.

When the number of labeled samples was reduced to 10%, which is the total label of 2400, while keeping the total amount of imagery fixed (24,000), DINO fine-tuning (FT) achieved the strongest results among the compared methods 78.5% accuracy, whereas DINO linear probing (LP) and ViT trailed (72.5% and 73% accuracy respectively) and the CNN achieved intermediate performance (77.5% accuracy). This outcome suggests that, under label-limited conditions, adapting the pre-trained DINO representations via end-to-end fine-tuning yields greater label efficiency than relying on a fixed encoder with a linear head (LP). The relatively weak performance of ViT in this case indicates that purely supervised transformers may be more label-hungry compared to self-supervised representations that can be adapted with limited supervision.

At 50% labeled data, ViT and DINO FT and LP became closely matched(85.7%, 85.2% and 85.4% accuracy respectively), with CNN emerging as a clear leader (87.2% accuracy). This behavior indicates that as more labels are revealed, the benefits of strong convolutional inductive biases reassert themselves, and linear separability of features produced by pre-trained transformers improves substantially. At 100% labeled data, CNN maintained the lead (90.2% accuracy), ViT ranked second (89.7% accuracy), and both DINO FT and DINO LP followed (88.6% and (86.4% accuracy), indicating that with abundant labels the supervised training dynamics of CNNs and ViTs capitalize on the full annotation budget more effectively than the fine-tuning or linear-probing regimes applied to DINO in this setup.

These observations highlight that label availability acts as a distinct bottleneck relative to total data. Self-supervised representations that are fine-tuned can be more label efficient at low-label regimes, whereas models with strong architectural priors (CNN) and supervised transformers (ViT) benefit more prominently as label density increases.

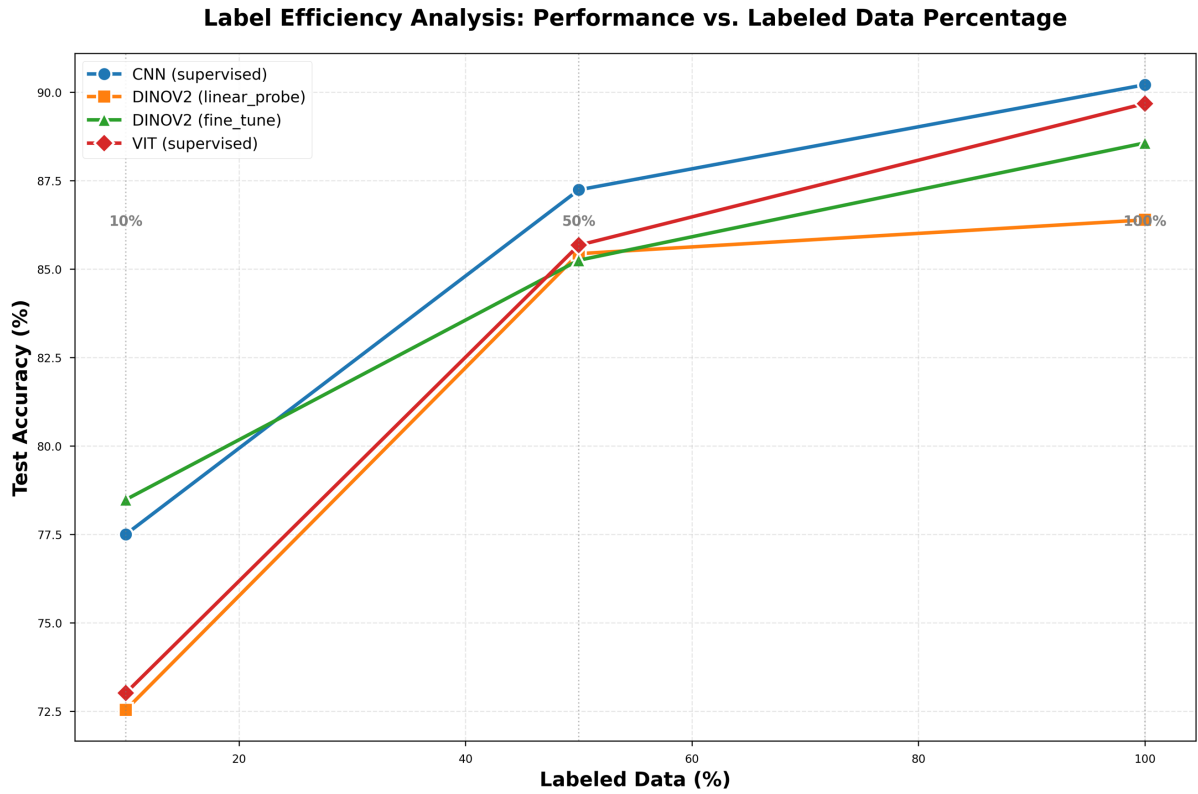


Figure 6.5: Label fraction comparison.

The overview in Figure 6.5 isolates the effect of label availability while keeping the image pool fixed. The separation at 10% labels confirms that label scarcity is a distinct bottleneck: end-to-end adaptation of pre-trained features (DINO FT) yields a stronger label-efficiency profile than either a linear head on frozen features (DINO LP) or fully supervised ViT training.

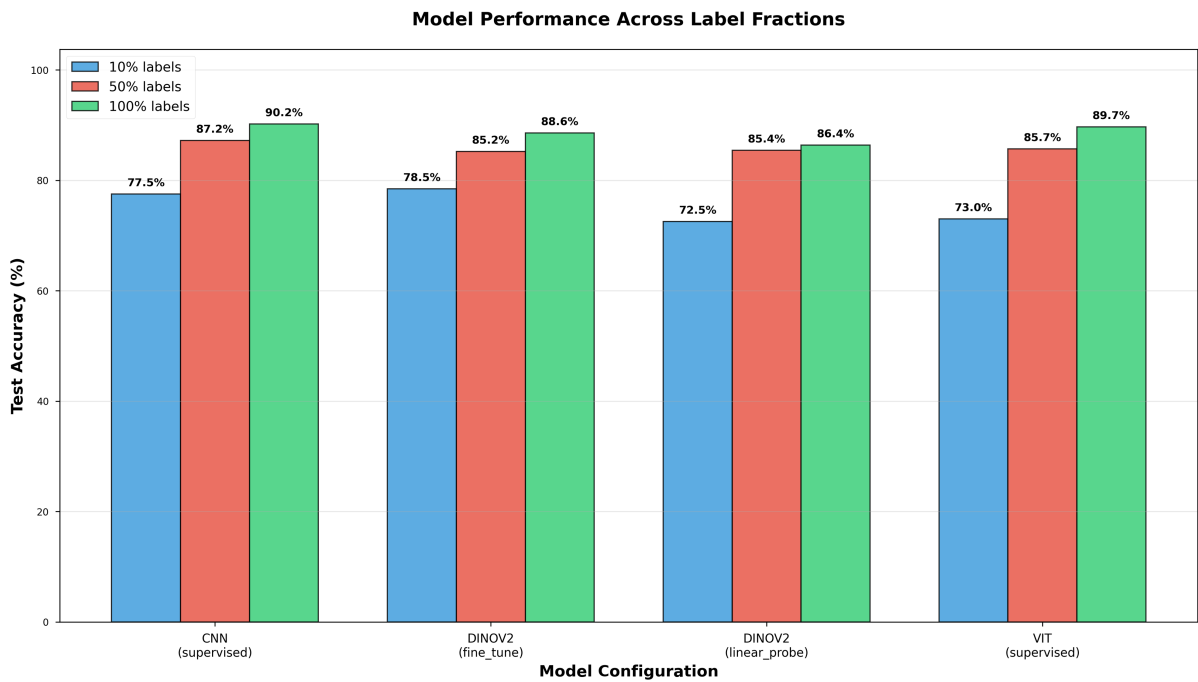


Figure 6.6: Label fraction comparison.

In Figure 6.6, performance rises sharply from 10% to 50% labeled data across methods, indicating that additional annotations are leveraged effectively. At 50%, the CNN emerges as the clear leader, ViT and the DINO variants (fine-tuning and linear probing) converge, suggesting that the linear separability of pre-trained transformer features improves as label density increases. At 100% labels, CNN sustains leadership and ViT ranks second, implying that abundant supervision favors models trained end-to-end in a fully supervised manner under this setup.

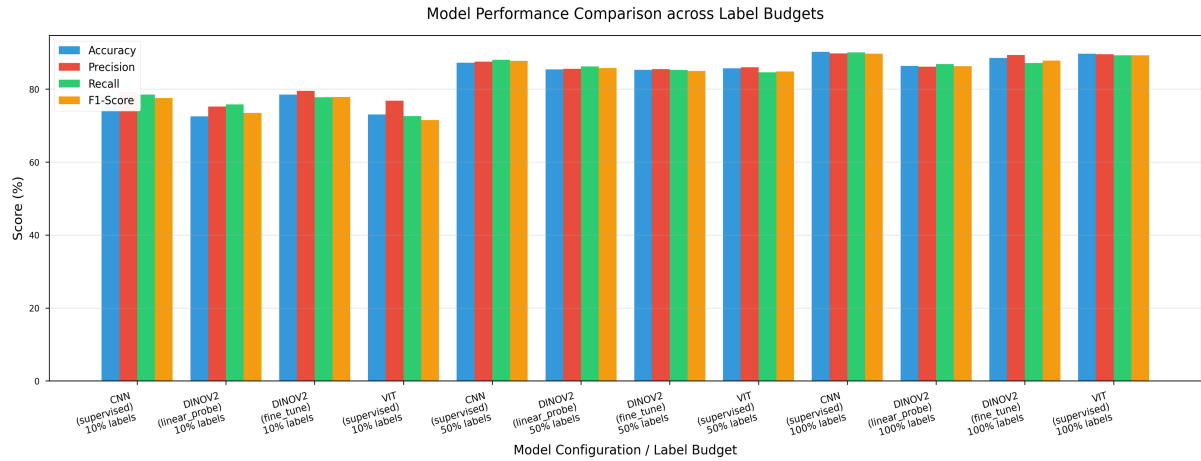


Figure 6.7: Label fraction comparison.

The Macro-F1 score (orange bars) in Figure 6.7 consistently rises across methods from 10% to 100% labeled data, demonstrating that increased label density benefits overall classification balance, including minority classes. The performance gap between DINO FT (approximately 77.5%) and DINO LP (approximately 74.5%) at 10% labels suggests that, in label-constrained settings, fine-tuning improves class decision boundaries beyond what a fixed encoder with a linear head can achieve. However, the subsequent convergence of ViT and DINO LP at 50% and 100% labels indicates that, with sufficient data, both fully supervised and frozen-feature pipelines learn comparably balanced and effective classifiers.

Detailed Experiment Results

Model	Regime	Budget	Train Acc	Test Acc	Time (s)
CNN	supervised	10%	88.22%	77.50%	347.6
DINOv2_FT	fine_tune	10%	89.00%	78.48%	172.3
DINOv2_LP	linear_probe	10%	82.26%	72.54%	59.8
ViT	supervised	10%	72.70%	73.02%	74.2
CNN	supervised	50%	96.44%	87.24%	737.2
DINOv2_FT	fine_tune	50%	84.16%	85.25%	408.9
DINOv2_LP	linear_probe	50%	89.02%	85.43%	708.4
ViT	supervised	50%	92.41%	85.67%	320.5
CNN	supervised	100%	97.51%	90.21%	1020.6
DINOv2_FT	fine_tune	100%	96.65%	88.56%	985.5
DINOv2_LP	linear_probe	100%	88.34%	86.39%	831.1
ViT	supervised	100%	96.52%	89.68%	658.7

Figure 6.8: Label fraction comparison.

6.1.3 Extreme Label Scarcity

This subsection evaluates the most challenging regimes (240 labels). Under the 1% label condition, DINO LP and DINO FT achieved the highest performance (56% accuracy), with CNN in third place (51.2%) and ViT last (22.6%). While self-supervised features from DINO clearly help when labels are scarce, CNN’s strong performance suggests that architectural priors such as locality and weight sharing still offer robustness under limited supervision. The ViT baseline, despite sharing the same backbone family as DINO, underperforms due to its reliance on extensive supervision and the absence of such inductive biases. The performance gap thus reflects both the benefits of self-supervised pretraining and the influence of dataset scale and bias, rather than a simple architecture-level hierarchy. Overall, the observed ranking reflects not only the benefits of self-supervised pretraining but also the influence of pretraining data scale and diversity, as DINOv2’s large and heterogeneous corpus likely contributes substantially to its advantage, which will be discussed in section 6.3 Comparative Discussion.

Practically, these findings suggest that when annotation budgets are extremely limited, leveraging high-quality self-supervised features with a lightweight classifier is an effective strategy, though architectures with strong inductive biases, such as CNNs, may also perform robustly. As label availability grows, fine-tuning and supervised training become increasingly beneficial, but under extreme scarcity, linear probing on pre-trained features, particularly those trained on large and diverse datasets, offers a stable and reliable approach.

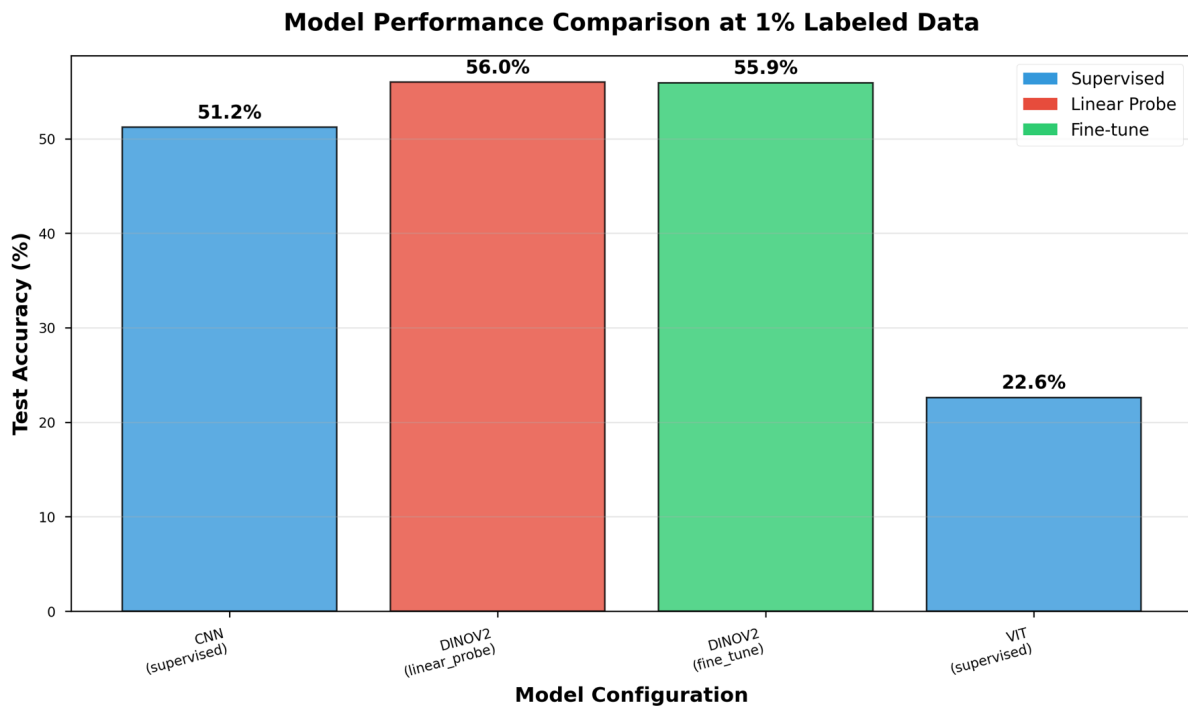


Figure 6.9: Extreme label scarcity comparison.

The comparison in Figure 6.9 focuses on the 1% label setting, where self-supervised features show a clear advantage. DINO LP and DINO FT perform similarly, suggesting that under extreme label scarcity, end-to-end fine-tuning neither substantially improves nor harms the utility of pre-trained embeddings.

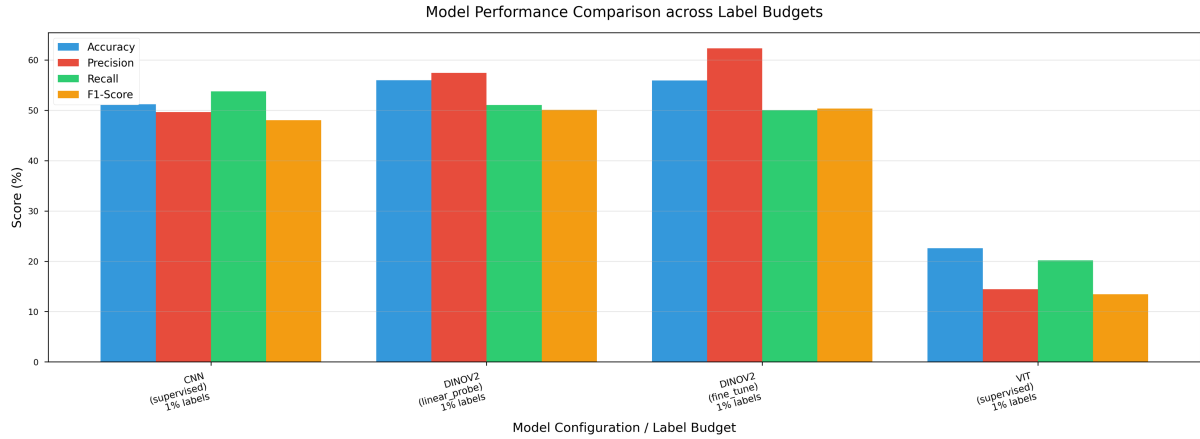


Figure 6.10: Extreme label scarcity comparison.

The macro-F1 in Figure 6.10 confirms that DINO LP (F1 50.5%) on high-quality features provides slightly better overall class balance under extreme label scarcity compared to DINO FT (F1 50.0%). While fine-tuning achieves substantially higher Precision (63.5%) , indicating that its positive predictions are more often correct , it exhibits slightly lower Recall (50.0%), meaning it captures fewer of the true positives. The resulting F1-score shows only a marginal improvement for the linear probe, suggesting that under severe label limitations, end-to-end fine-tuning introduces a modest trade-off: higher confidence in predictions at the cost of reduced coverage, potentially making the classifier more conservative. In contrast, the fixed-feature linear probe maintains a more balanced trade-off between Precision and Recall, which can be advantageous when labeled data is extremely scarce.

Model Summary Table: 1% Labels

Model	Test Acc	Val Acc	Model Size	Train Time	Epochs (T/P)
CNN	51.2%	51.8%	202.4 MB	2.4m (144s)	8 / 75
DINOv2_FT	55.9%	56.8%	330.6 MB	1.1m (67s)	7 / 75
DINOv2_LP	56.0%	56.6%	330.6 MB	35s	3 / 60
ViT	22.6%	22.6%	982.3 MB	1.3m (79s)	8 / 75

Figure 6.11: Extreme label scarcity comparison.

Taken together, the extreme-label results suggest a practical guideline: for agricultural datasets with strong local structure, when annotation budgets are minimal, leveraging pre-trained features, either via a frozen encoder or architectures with strong inductive biases, offers a robust starting point; as labels increase, end-to-end training becomes increasingly beneficial.

6.1.4 Hyperparameter Sensitivity

Sensitivity to the warmup schedule revealed substantial interaction between optimization dynamics and label availability. With a warmup of 15 epochs, DINO FT led at 10% labeled data, achieving an accuracy of approximately 78.5%. In contrast, a shorter warmup of 3 epochs reduced the 10% performance to

approximately 69.7%, placing DINO FT second. This sensitivity indicates that longer warmup can stabilize adaptation of the pre-trained encoder under limited supervision, likely by moderating the magnitude of early gradient updates and preventing detrimental drift from the pre-trained feature space.

At 100% labeled data, a different pattern emerged: a 3-epoch warmup resulted in DINO FT achieving roughly 90.4%, surpassing the CNN at 90.2% and outperforming the 10-epoch warmup that placed DINO FT around 88.6%. This reversal suggests that, with abundant labels, faster engagement of the full learning rate facilitates more aggressive fitting and better use of the larger supervision signal. Consequently, the optimal warmup is not universal but depends on label density: longer warmup appears preferable in scarce-label regimes to preserve pre-trained structure, whereas shorter warmup becomes advantageous when labels are plentiful and rapid adaptation is beneficial.

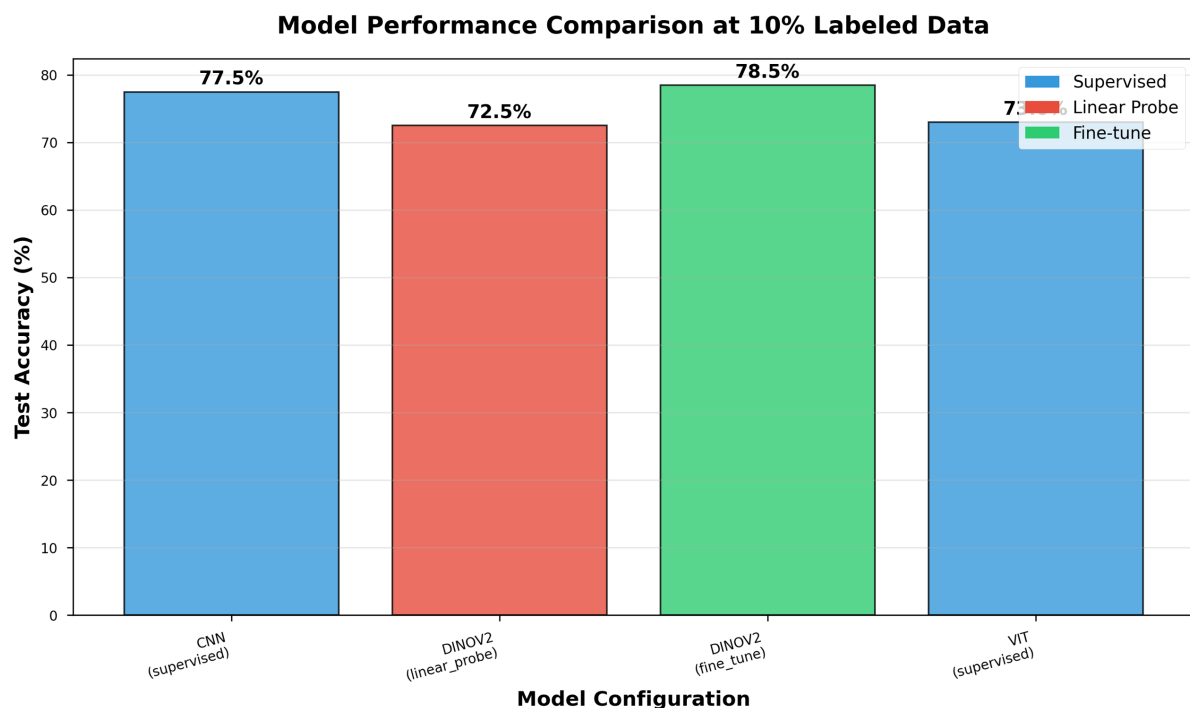


Figure 6.12: Hyperparameter sensitivity in DINOv2 early stop 15 used with 10% data.

The configuration in Figure 6.12 demonstrates that a longer warmup provides smoother optimization under 10% labels, likely by constraining early updates and preventing catastrophic deviation from the pre-trained manifold.

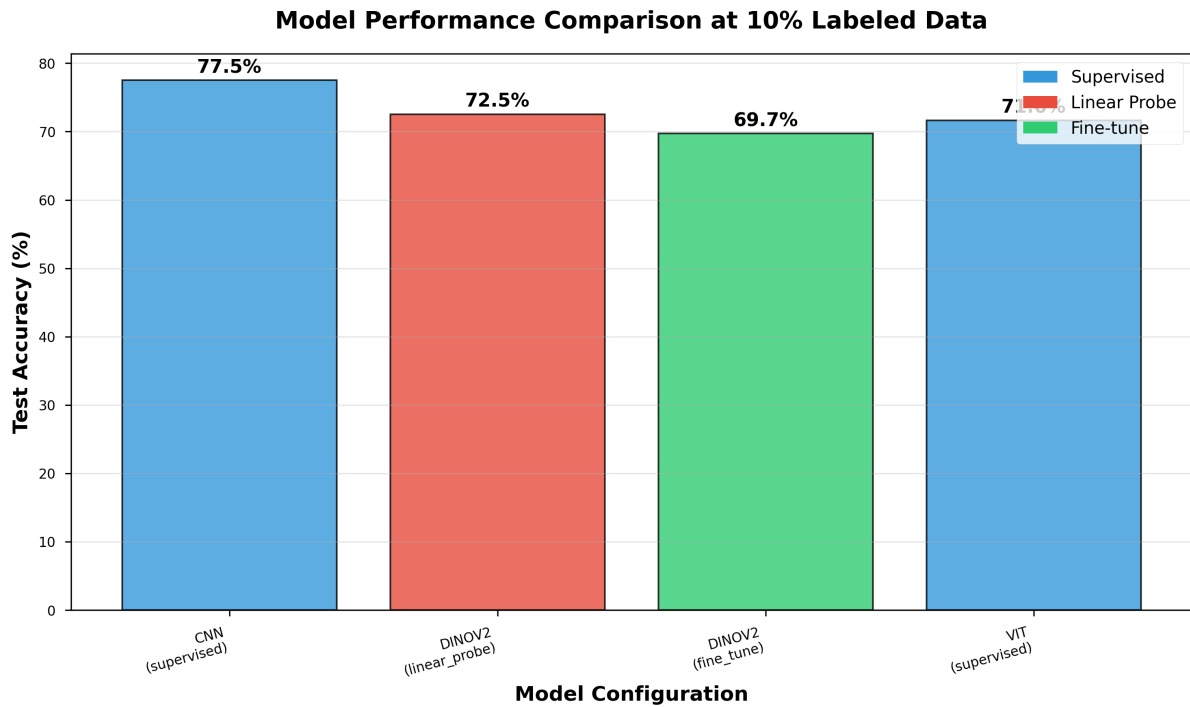


Figure 6.13: Hyperparameter sensitivity in DINOv2 early stop 3 used with 10% data.

By contrast, Figure 6.13 shows that a short warmup yields lower performance at 10% labels, consistent with more aggressive early learning rate exposure harming adaptation with scarce supervision.

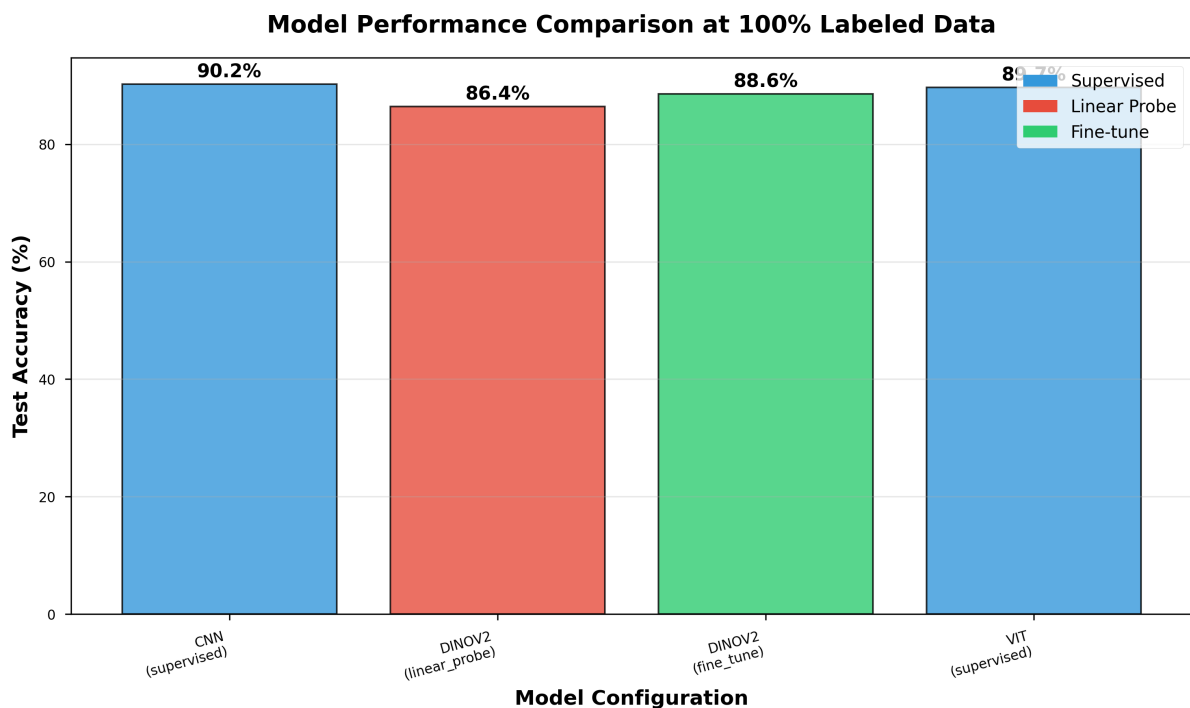


Figure 6.14: Hyperparameter sensitivity in DINOv2 early stop 15 used with 100% data.

Under full labels, Figure 6.14 indicates that a long warmup may constrain learning unnecessarily, leading to slightly reduced final performance compared to a shorter warmup.

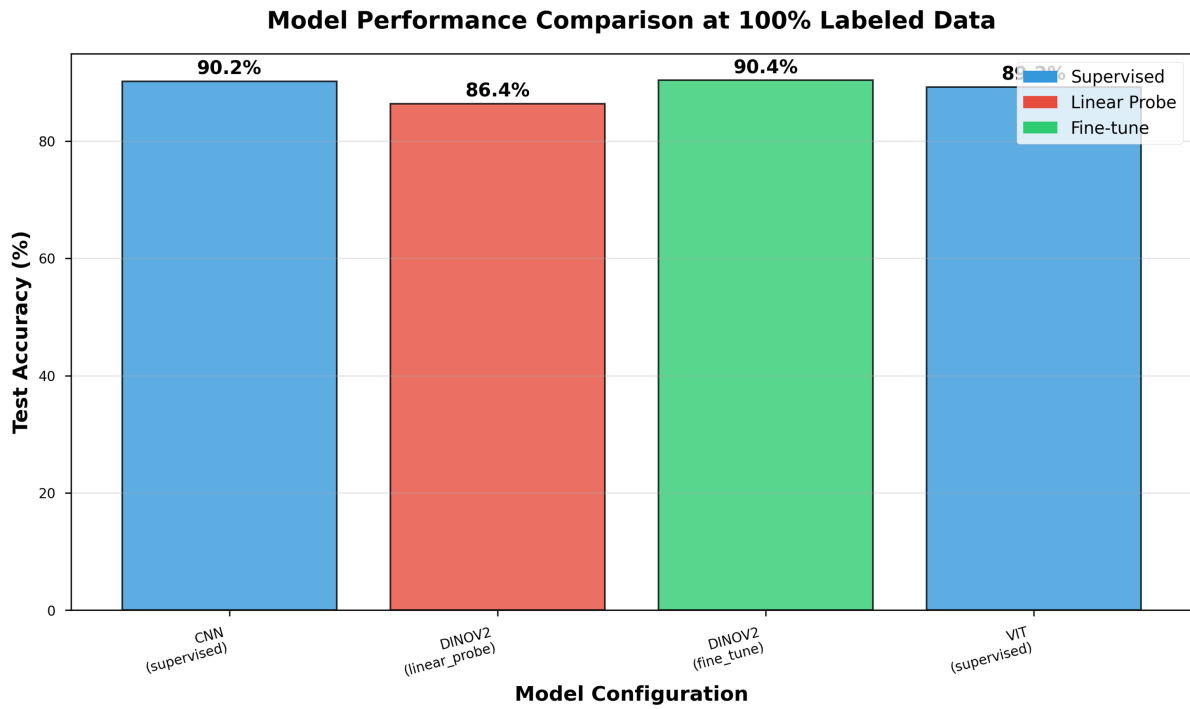


Figure 6.15: Hyperparameter sensitivity in DINOv2 early stop 3 used with 100% data.

Finally, Figure 6.15 illustrates that with 100% labels a shorter warmup enables faster and ultimately better convergence, aligning with the observation that abundant supervision benefits from more rapid engagement of the learning rate.

6.2 Quantization and Deployment Efficiency

6.2.1 Quantization - Data Fraction

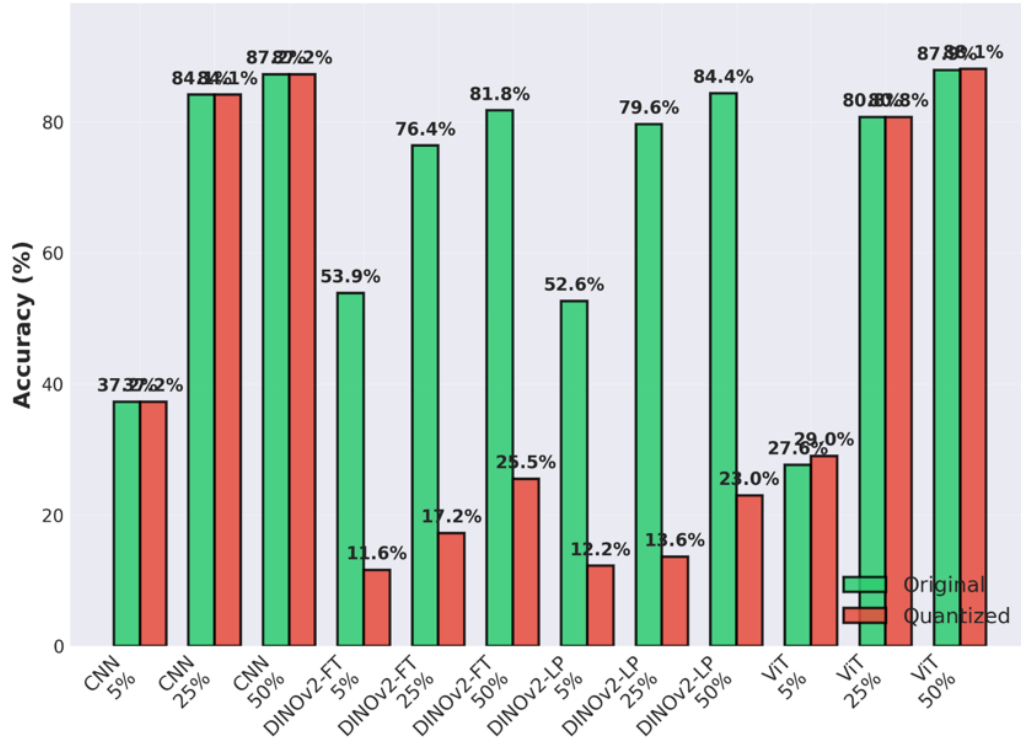


Figure 6.16: Quantization accuracy comparison.

It was observed that dynamic quantization affected architectures differently across data fractions. CNN accuracy remained essentially unchanged before and after quantization, indicating that the convolutional baseline is robust to reduced precision under the evaluated settings. In contrast, DINO FT exhibited more sensitivity when data were scarce, with noticeable accuracy degradation at lower data fractions, while recovering as data increased. DINO LP also showed a consistent decrease in accuracy under quantization, suggesting that the linear head on frozen features is more susceptible to quantization noise. ViT has shown that quantization can be effective for ViT supervised models, but it is important to note that the impact of quantization on accuracy can vary depending on the architecture and the specific application.

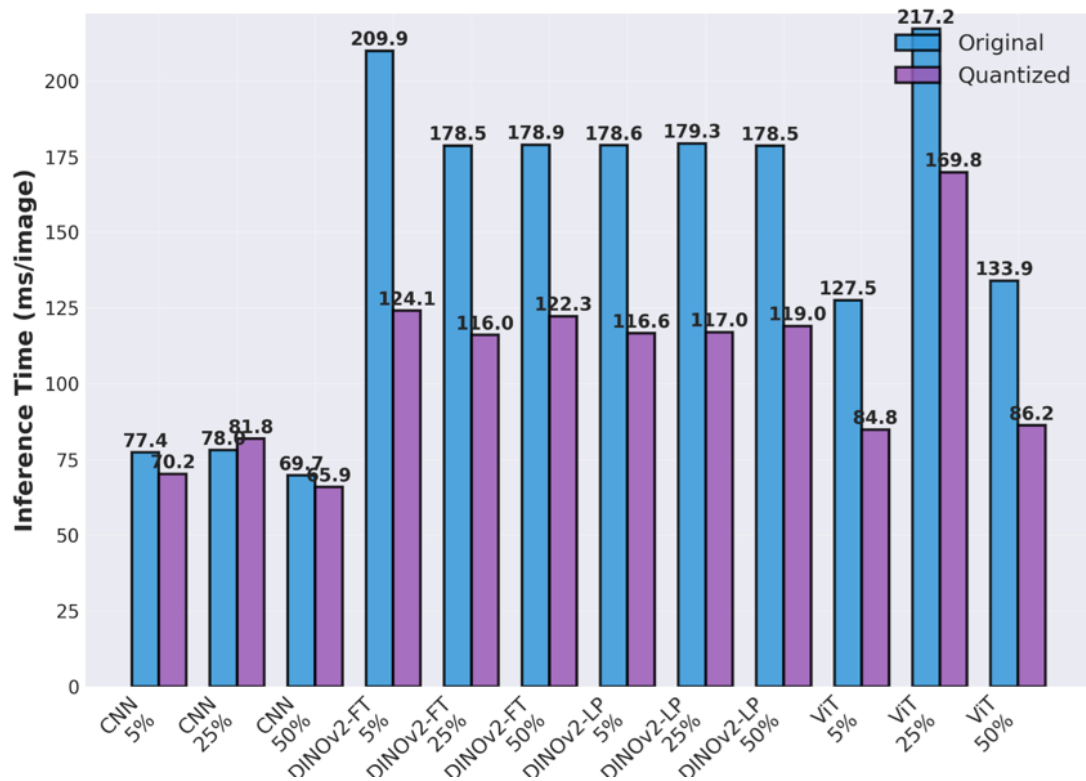


Figure 6.17: Quantization inference comparison.

The inference-time measurements indicated that CNN experienced little to no change in speed, consistent with the observation that speed reduction was not altered by quantization for this architecture. For DINO FT, speed improvements were realized but tended to plateau, with reductions resembling those seen at intermediate fractions (e.g., around the 50% setting). LP followed a similar speed profile to FT. ViT has shown drastic speed improvements under quantization, making it a promising candidate for deployment in resource-constrained environments, especially when we consider it has also showed strong accuracy retention.

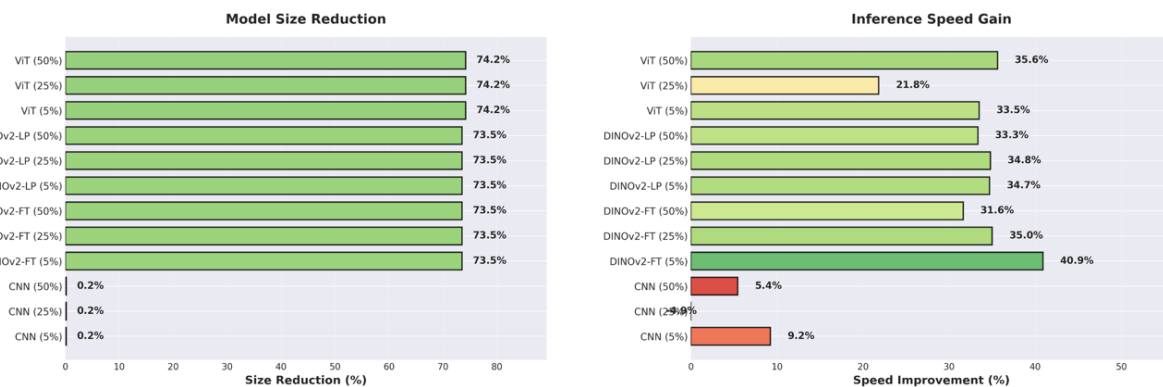


Figure 6.18: Quantization speed and size comparison.

In Figure 6.18, the model size and inference speed results illustrate differential impacts of quantization across architectures. The size reduction achieved by quantizing ViT-based models (DINO and ViT) is substantial and consistent, clustering around 73.5% to 74.2% across all data fractions (5%, 25%, 50%).

The compression ratios for DINO FT, DINO LP, and ViT are nearly identical, reflecting that quantization primarily affects the large, shared ViT encoder backbone. In contrast, the CNN baseline exhibits only a minimal size reduction of 0.2% across all data budgets, consistent with its smaller parameter count and structural robustness.

Inference speed results highlight a significant advantage for the ViT-based models compared to CNN. The DINO variants consistently achieve speed improvements in the range of 31.6% to 40.9%, with DINO FT at 5% labels showing the maximum gain (40.9%). The supervised ViT also shows strong but slightly less consistent improvements (21.8% to 35.6%). By comparison, the CNN baseline gains only modestly, ranging from 5.4% to 9.2%.

6.2.2 Quantization - Label Fraction

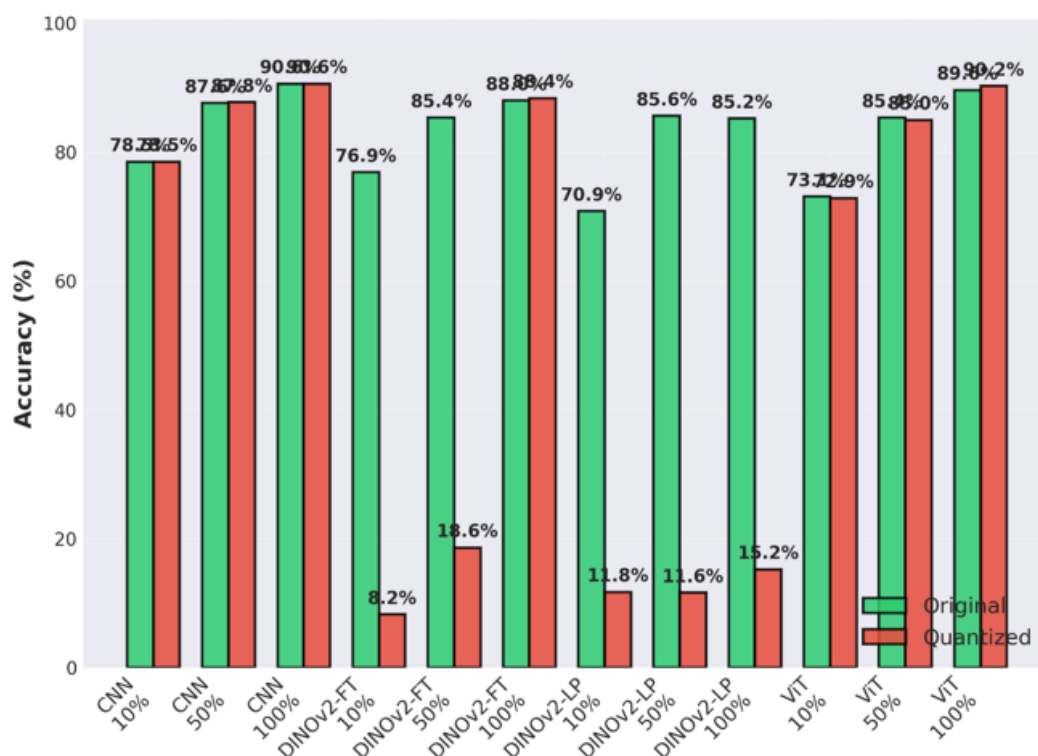


Figure 6.19: Quantization accuracy comparison.

Across label fractions, the impact of quantization on accuracy diverged by method. CNN performance remained effectively unchanged after quantization at each label setting, indicating high robustness. DINO FT, except at the 100% label condition, exhibited poorer accuracy under quantization, particularly at low-label regimes where end-to-end adaptation is fragile; at 100% labels the method recovered and became competitive. LP showed a consistent decrease in accuracy with quantization across label fractions, reflecting the vulnerability of linear classification on frozen features to quantization noise. ViT shows clear accuracy retention here in label scarcity as well.

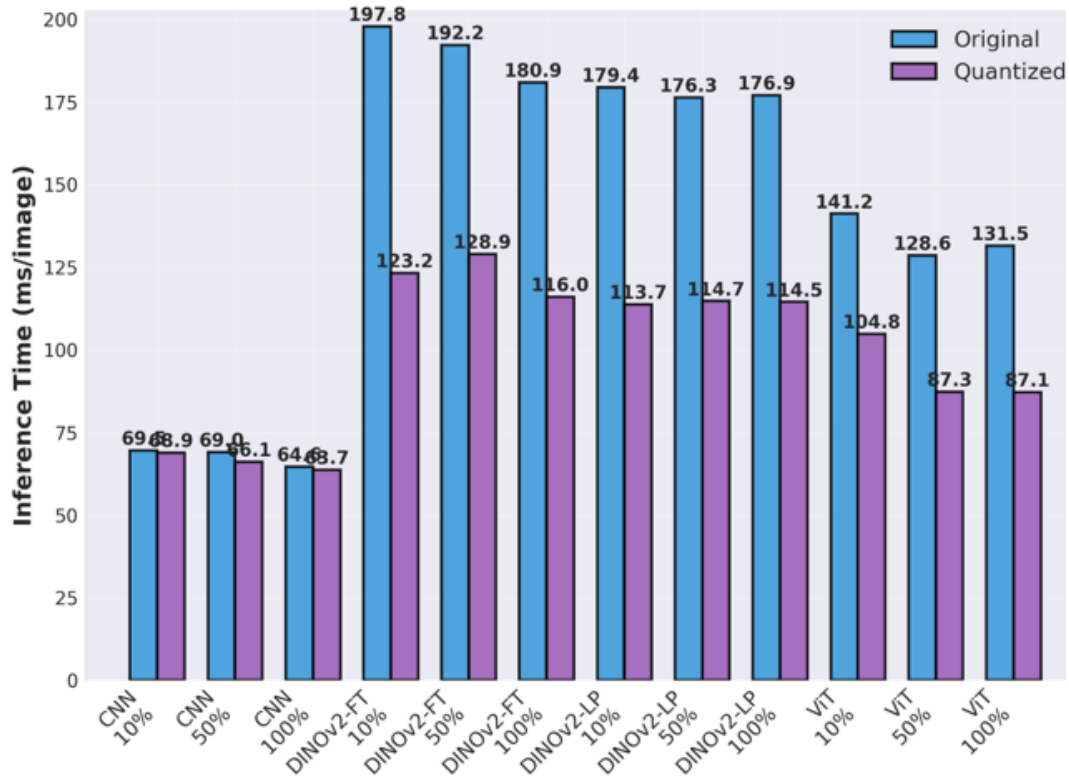


Figure 6.20: Quantization inference comparison.

The inference-time comparison indicated that CNN speed gains were minimal, aligning with observations from the data-fraction analysis. For DINO FT, speed reductions under quantization were evident but similar to the gains observed at 50% label setting, suggesting saturation in achievable gains from reduced precision. DINO FT and LP and ViT share similar runtime characteristics due to their common backbone, although supervised ViT is slightly faster than both LP and FT, potentially related to the size of the pretraining corpus.

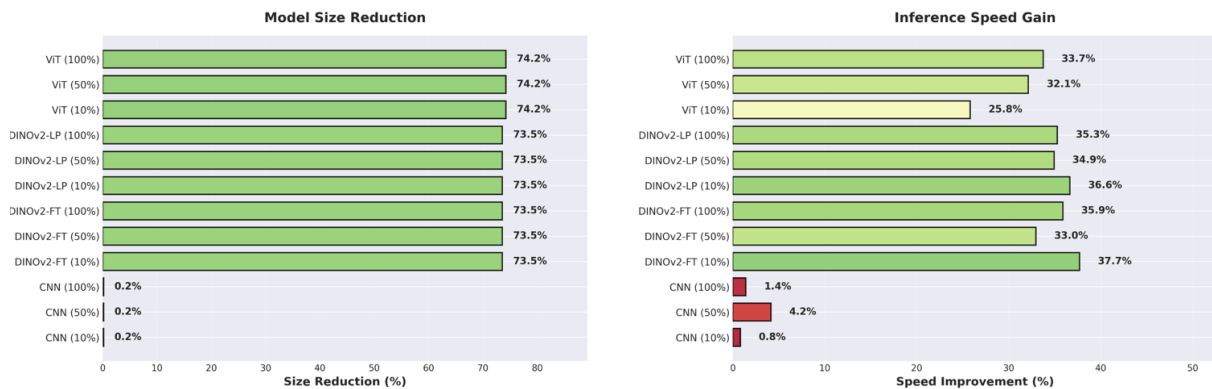


Figure 6.21: Quantization speed and size comparison.

Model size results across label fractions show that DINO LP and FT achieved essentially the same reduction (73.5%), consistent with quantizing the shared encoder. The CNN baseline experienced only minimal size reduction (0.2%) without discernible accuracy loss, very smiliar to what we have discussed in the previous subsection.

6.3 Comparative Discussion

The empirical results collectively indicate that representation learning strategy and architectural priors interact strongly with data and label availability, as well as with deployment constraints such as precision reduction. Under data and label scarcity, self-supervised features derived from DINO provided a measurable advantage in label efficiency, particularly when used either with linear probing (LP) in the most extreme regimes or with end-to-end fine-tuning (FT) when a modest set of labels was available. In contrast, purely supervised ViT training required more data to realize its capacity, while convolutional networks offered robust performance at moderate and higher data/label fractions due to their inductive biases.

In the lowest data fractions, convolutional inductive biases translated into strong baselines, whereas DINO-derived representations remained competitive and, in some cases, superior when labels were scarce. As more data and labels became available, the CNN sustained a lead in accuracy on this dataset, with ViT catching up and DINO FT/LP showing diminishing relative advantage, consistent with the expectation that abundant supervision favors fully supervised training dynamics.

Under extreme label scarcity, linear probing on DINO features attained the most stable performance, followed closely by DINO FT, indicating that the separability afforded by high-quality self-supervised embeddings can outweigh the benefits of end-to-end adaptation when annotations are minimal. The gap between LP and FT in this regime suggests that the variance introduced by full-parameter optimization may not be compensated by the very limited supervision signal.

Sensitivity to warmup revealed substantial interaction between optimization dynamics and label availability. The reported results were obtained using textbfearly stopping, which prevents overfitting and ensures the final accuracy is achieved efficiently. With a 15 epoch warmup, DINO FT led at 10% labeled data (accuracy 78.5%), indicating that longer warmup stabilizes adaptation of the pre-trained encoder under limited supervision by moderating early gradient updates and preventing detrimental feature drift. In contrast, at 100% labeled data, a 3 epoch warmup resulted in DINO FT achieving a high accuracy of 90.4% (surpassing the CNN), demonstrating that with abundant labels, shorter warmup is advantageous as it allows faster engagement of the full learning rate to exploit the large supervision signal. Hence, hyperparameters that are optimal in one supervision regime may be suboptimal in another.

Quantization experiments emphasized deployment trade-offs. CNN accuracy was essentially unchanged after quantization and speed reductions remained stable, making the convolutional baseline an attractive option for resource-constrained or latency sensitive platforms such as drone deployment. DINO FT exhibited accuracy degradation under quantization at low-label conditions and recovered at 100% labels, while LP showed a consistent accuracy decrease; in both cases, model size reductions were comparable owing to the shared encoder. The protected quantization variant preserved accuracy across most conditions and delivered strong size and speed improvements, indicating a favorable balance between compression and performance when such protective strategies are available.

Training efficiency and early stopping The reported results were obtained using early stopping, which terminated training once validation performance plateaued, even though runs were scheduled for 60, 75 epochs depending on the model. As shown in Table 6.1, actual training epochs (T) were often substantially lower than planned epochs (P), DINO FT trained for 16 of 75 planned epochs at 5% data, or CNN trained for 14 of 75 epochs at 50% labeled data. This early termination reduced total training time,

Table 6.1: Planned vs. actual training epochs, train time, and test accuracy for each model under varying data/label availability. T/P indicates Trained/Planned epochs.

<i>Model</i>	Test Acc	Train Time	T/P Epochs	Data/Label Fraction
CNN	45.7%	39s	5/75	5% data
DINOv2 FT	57.5%	53s	16/75	5% data
DINOv2 LP	56.9%	28s	8/60	5% data
ViT	20.7%	22s	5/75	5% data
CNN	90.0%	3.2m	17/75	25% data
DINOv2 FT	83.7%	1.7m	16/75	25% data
DINOv2 LP	84.0%	4.7m	53/60	25% data
ViT	85.3%	1.7m	13/75	25% data
CNN	89.5%	7.0m	19/75	50% data
DINOv2 FT	85.7%	3.9m	24/75	50% data
DINOv2 LP	86.1%	6.5m	52/60	50% data
ViT	88.4%	3.2m	16/75	50% data
CNN	51.2%	2.4m	8/75	1% labels
DINOv2 FT	55.9%	1.1m	7/75	1% labels
DINOv2 LP	56.0%	35s	3/60	1% labels
ViT	22.6%	1.3m	8/75	1% labels
CNN	87.2%	12.3m	14/75	50% labels
DINOv2 FT	85.2%	6.8m	24/75	50% labels
DINOv2 LP	85.4%	11.8m	50/60	50% labels
ViT	85.7%	5.3m	17/75	50% labels
CNN	90.2%	17.0m	20/75	100% labels
DINOv2 FT	88.6%	16.4m	42/75	100% labels
DINOv2 LP	86.4%	13.9m	50/60	100% labels
ViT	89.7%	11.0m	26/75	100% labels

prevented overfitting, and ensured that reported accuracies reflect effective utilization of the available labels rather than unnecessary computation. Consequently, the observed efficiency differences across model families such as, faster convergence of CNNs at higher label fractions or DINO LP at very low labels, arise from both architectural inductive biases and the interaction with early stopping.

Overall, the results support the following interpretation. When label budgets are minimal, relying on self-supervised features with a lightweight classifier is a robust and data-efficient choice. With moderate labels, DINO FT effectively exploits supervision while retaining much of the self-supervised advantage, provided the optimization schedule is appropriately tuned for the label density. As labels become plentiful, CNNs and supervised ViTs capitalize on end-to-end training and often yield the highest accuracy. For deployment scenarios prioritizing inference speed and energy efficiency, quantized CNNs (and protected quantized variants of transformer-based models) provide a practical path with minimal accuracy loss. When inference latency is less critical, DINO-based approaches remain compelling due to their strong performance in scarcity regimes and competitive results after careful hyperparameter tuning.

Summary The interaction between supervision level, model family, and deployment constraints suggests a regime-dependent strategy:

- (i) Extreme scarcity favors linear probing on self-supervised features.
- (ii) Low to moderate labels benefit from carefully tuned fine-tuning of self-supervised encoders, requiring a 15-epoch or similar warmup to maintain feature fidelity.
- (iii) Abundant labels favor fully supervised CNN/ViT pipelines with a 3-epoch or similar warmup to maximize rapid adaptation.
- (iv) For constrained hardware, CNNs, especially with or without quantization or protected quantized transformers, offer balanced accuracy, latency, and size trade-offs.

7 Challenges

Conducting a comparative study across multiple architectures and supervision regimes presented significant computational and methodological challenges. In total, twenty-eight independent training runs were required: four model families (CNN, DINOv2 fine-tuned, DINOv2 linear probe, and ViT) evaluated under seven different data and label regimes (5%, 25%, and 50% data; 1%, 10%, 50%, and 100% labeled samples). This multiplicative setup quickly escalated the total computational demand, especially since transformer-based and self-supervised models such as DINOv2 are substantially heavier than convolutional baselines. Consumer-grade GPUs proved insufficient for sustained experiments, requiring cloud-based GPU hosting with hourly billing constraints. This computational overhead limited both the breadth of hyperparameter exploration and the number of reproducibility trials that could be reasonably executed.

Hyperparameter sensitivity was another major challenge, particularly for self-supervised fine-tuning. Unlike purely supervised CNNs, which exhibit relatively smooth loss landscapes and predictable learning dynamics, self-supervised encoders are highly sensitive to optimizer configuration, warmup scheduling, and layer-wise learning rates. Small deviations can lead to representational collapse or catastrophic forgetting of pre-trained features. While systematic optimization techniques such as Optuna or grid searches could have improved performance, financial prevented full-scale hyperparameter sweeps. Instead, a pragmatic balance was struck between reproducibility and feasibility by adopting empirically reasonable configurations informed by prior literature.

Data-related issues further complicated the experimental process. The primary dataset, although clean and high-resolution, favored convolutional models because it lacked the visual variability (lighting, occlusion, viewpoint diversity) that often benefits feature-rich transformer backbones. When experimenting with noisier and more heterogeneous datasets, severe class imbalance and labeling inconsistencies emerged, degrading both training stability and evaluation reliability. Nevertheless, such datasets are the dataset, where SSL especially shines.

Moreover, ensuring fair comparison across model families was important. Each architecture entails distinct inductive biases, input normalization pipelines, and training dynamics, which complicates attempts to isolate the effect of supervision level alone. Balancing these structural differences within a unified experimental framework demanded numerous pilot runs and fine-grained monitoring of convergence behavior. Ultimately, these constraints shaped the experimental design philosophy, prioritizing methodological transparency and replicability over exhaustive parameter tuning.

These challenges collectively underscore the tension between experimental comprehensiveness, computational feasibility, and ecological realism in applied computer vision. Large-scale comparisons across architectures, especially under multiple supervision regimes, are inherently resource-intensive and require trade-offs between hyperparameter optimization depth and the diversity of tested conditions.

8 Conclusion

This study conducted a systematic comparison between self-supervised and supervised learning paradigms across varying levels of data and label availability, using a challenging agricultural benchmark, the Crop Pest and Disease Detection Dataset subsection 5.1.2. The analysis encompassed four architectures, CNN, ViT, DINOv2 fine-tuned, and DINOv2 with linear probing, across seven supervision regimes ranging from extreme scarcity to full supervision. Through this comprehensive evaluation, the study highlights not only model-level differences in accuracy and convergence but also the critical interaction between data regime, representation quality, and deployment constraints.

Overall, the findings confirm that self-supervised pretraining provides substantial resilience under low-label and low-data conditions. Both DINOv2 fine-tuned (FT) and linear probing (LP) models consistently outperformed or closely matched fully supervised baselines when annotations were limited. In extreme scarcity conditions, linear probing achieved competitive accuracies while maintaining remarkable stability, reflecting the inherent separability and discriminative power of DINOv2’s pre-trained embeddings. These results reinforce the premise that high-quality self-supervised representations act as strong priors that reduce the dependency on labeled supervision, a finding well aligned with recent literature in representation learning and few-shot adaptation.

Interestingly, despite the dataset’s strong bias toward local pattern recognition, such as, leaf texture, lesion shape, and color gradients, favoring convolutional architectures, the DINO-based approaches demonstrated remarkable robustness. This is particularly notable given that transformer backbones typically excel in capturing global structure rather than localized textures. The fact that DINOv2 retained competitive accuracy even in this CNN-favored context underscores its ability to internalize both global and local dependencies via its self-distillation training regime. This capacity is expected to generalize even more strongly to datasets that demand global reasoning, such as those with complex scenes or weak object localization.

From a methodological perspective, these results also emphasize that the utility of self-supervised models extends beyond accuracy metrics. DINOv2, provided not only data efficiency but also lower variance under early stopping and smaller overfitting risk. In contrast, purely supervised ViT required larger datasets to unlock its potential, even if it is the same backbone. While CNNs remained the most stable and efficient option when abundant labels and clean, structured data were available. Hence, the optimal strategy is regime-dependent: self-supervision dominates in scarcity; convolutional inductive biases prevail in abundance.

From a deployment standpoint, quantization experiments revealed that CNNs maintained superior stability and accuracy under compression, also stayed fastest inference and one of the lowest model size. In contrast, the DINOv2-based models suffered notable degradation when subjected to dynamic quantization. This outcome is consistent with the known limitations of dynamic quantization for transformer architectures: it primarily compresses linear layers and does not adequately handle layer normalization, attention projections, or activation statistics, which are crucial for maintaining representational integrity in self-supervised encoders. The supervised ViT, proved more robust than DINOv2 under this form of quantization. These findings suggest that while CNNs remain the most deployment-ready choice under simple post-training quantization, alternative techniques, such as quantization-aware

training, mixed-precision fine-tuning, or protected quantization, could potentially preserve the representational quality of self-supervised transformers more effectively. This limitation highlights an important avenue for future work, especially for edge computing and agricultural robotics contexts where memory, latency, and energy constraints are paramount.

Finally, the broader contribution of this thesis lies in clarifying the conditions under which self-supervised transformer (DINOv2) genuinely offer an advantage. They are not a universal replacement for convolutional networks but rather a complementary paradigm that excels in settings characterized by limited annotation, diverse or messy data, and the need for rapid adaptation to unseen visual domains. With continued improvements in pre-training strategies, such as the recently introduced DINOv3 [70], which enhances multi-crop self-distillation and regularization, future research may achieve even higher robustness and generalization.

In conclusion, this work demonstrates that while classical CNNs remain a strong and efficient baseline for structured visual tasks, self-supervised transformer models like DINOv2 already bridge much of the gap, achieving competitive or superior performance under data and label scarcity. As the field advances toward increasingly generalizable and data-efficient vision systems, self-supervised learning, anchored in scalable pre-training and domain adaptation, will likely form the foundation of the next generation of agricultural and remote-sensing AI models.

References

- [1] V. K. Vishnoi, K. Kumar, and B. Kumar, “Plant disease detection using computational intelligence and image processing,” *Journal of Plant Diseases and Protection*, vol. 128, no. 1, pp. 19–53, Feb. 1, 2021, ISSN: 1861-3837. DOI: 10.1007/s41348-020-00368-0. [Online]. Available: <https://doi.org/10.1007/s41348-020-00368-0>.
- [2] R. Thangaraj, S. Anandamurugan, P. Pandiyan, and V. K. Kaliappan, “Artificial intelligence in tomato leaf disease detection: A comprehensive review and discussion,” *Journal of Plant Diseases and Protection*, vol. 129, no. 3, pp. 469–488, Jun. 1, 2022, ISSN: 1861-3837. DOI: 10.1007/s41348-021-00500-8. [Online]. Available: <https://doi.org/10.1007/s41348-021-00500-8>.
- [3] A. Bouguettaya, H. Zarzour, A. Kechida, and A. M. Taberkit, “A survey on deep learning-based identification of plant and crop diseases from UAV-based aerial images,” *Cluster Computing*, vol. 26, no. 2, pp. 1297–1317, Apr. 2023, ISSN: 1386-7857, 1573-7543. DOI: 10.1007/s10586-022-03627-x. [Online]. Available: <https://link.springer.com/10.1007/s10586-022-03627-x> (visited on 07/10/2025).
- [4] N. Wang, G. W. Sundin, L. D. L. Fuente, *et al.*, “Key challenges in plant pathology in the next decade,” *Phytopathology*®, vol. 114, no. 5, pp. 837–842, 2024, PMID: 38815216. DOI: 10.1094/PHYTO-04-24-0137-KC. [Online]. Available: <https://doi.org/10.1094/PHYTO-04-24-0137-KC>.
- [5] M. Mukhiddinov, A. B. Abdusalomov, and J. Cho, “A wildfire smoke detection system using unmanned aerial vehicle images based on the optimized yolov5,” *Sensors*, vol. 22, no. 23, 2022, ISSN: 1424-8220. DOI: 10.3390/s22239384. [Online]. Available: <https://www.mdpi.com/1424-8220/22/23/9384>.
- [6] H. Huang, J. Deng, Y. Lan, A. Yang, X. Deng, and L. Zhang, “A fully convolutional network for weed mapping of unmanned aerial vehicle (uav) imagery,” *PLOS ONE*, vol. 13, pp. 1–19, Apr. 2018. DOI: 10.1371/journal.pone.0196302. [Online]. Available: <https://doi.org/10.1371/journal.pone.0196302>.
- [7] P. Tresson, M. Dumont, M. Jaeger, *et al.*, “Self-supervised learning of vision transformers for digital soil mapping using visual data,” *Geoderma*, vol. 450, p. 117056, 2024, ISSN: 0016-7061. DOI: <https://doi.org/10.1016/j.geoderma.2024.117056>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0016706124002854>.
- [8] X. Wanyan, S. Seneviratne, S. Shen, and M. Kirley, *Extending global-local view alignment for self-supervised learning with remote sensing imagery*, 2024. arXiv: 2303.06670 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2303.06670>.
- [9] X. Chen, S. Xie, and K. He, *An empirical study of training self-supervised vision transformers*, 2021. arXiv: 2104.02057 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2104.02057>.

- [10] M. Caron, H. Touvron, I. Misra, *et al.*, “Emerging properties in self-supervised vision transformers,” in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 9630–9640. DOI: 10.1109/ICCV48922.2021.00951.
- [11] K. Shaheed, I. Qureshi, F. Abbas, *et al.*, “EfficientRMT-Net—An Efficient ResNet-50 and Vision Transformers Approach for Classifying Potato Plant Leaf Diseases,” *Sensors*, vol. 23, no. 23, p. 9516, Nov. 30, 2023, ISSN: 1424-8220. DOI: 10.3390/s23239516. [Online]. Available: <https://www.mdpi.com/1424-8220/23/23/9516> (visited on 07/10/2025).
- [12] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943.
- [13] F.-F. Li, J. Johnson, and S. Yeung, *Cs231n: Convolutional neural networks for visual recognition*, <https://cs231n.github.io/>, <http://cs231n.stanford.edu/>, 2017.
- [14] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, pp. 386–408, 1958.
- [15] M. L. Minsky and S. A. Papert, *Perceptrons: An Introduction to Computational Geometry*, 1st. Cambridge, MA: MIT Press, 1969.
- [16] G. Bíró. “The first ai winter: When early promises met reality.” (Jul. 2024), [Online]. Available: <https://www.birow.com/az-also-ai-tel>.
- [17] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986. DOI: 10.1038/323533a0.
- [18] H. Robbins and S. Monro, “A stochastic approximation method,” *Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.
- [19] X. Glorot, A. Bordes, and Y. Bengio, “Deep sparse rectifier neural networks,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 15, PMLR, 2011, pp. 315–323. [Online]. Available: <https://proceedings.mlr.press/v15/glorot11a/glorot11a.pdf>.
- [20] D. H. Hubel and T. N. Wiesel, “Receptive fields of single neurones in the cat’s striate cortex,” *The Journal of Physiology*, vol. 160, no. 1, pp. 106–154, 1962. DOI: 10.1113/jphysiol.1962.sp006837.
- [21] K. Fukushima, “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position,” *Biological Cybernetics*, vol. 36, no. 4, pp. 193–202, 1980. DOI: 10.1007/BF00344251.
- [22] Y. LeCun, B. Boser, J. S. Denker, *et al.*, “Backpropagation applied to handwritten zip code recognition,” *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989. DOI: 10.1162/neco.1989.1.4.541.
- [23] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [24] A. A. M. Omer, “Image Classification Based on Vision Transformer,” *Journal of Computer and Communications*, vol. 12, no. 04, pp. 49–59, 2024, ISSN: 2327-5219, 2327-5227. DOI: 10.4236/jcc.2024.124005. [Online]. Available: <https://www.scirp.org/journal/doi.aspx?doi=10.4236/jcc.2024.124005> (visited on 07/10/2025).

- [25] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097–1105.
- [26] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2009, pp. 248–255.
- [27] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, Jan. 2014, ISSN: 1532-4435.
- [28] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *International Conference on Learning Representations (ICLR)*, 2015, arXiv:1409.1556.
- [29] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, 2015. arXiv: 1512.03385 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1512.03385>.
- [30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, *An image is worth 16x16 words: Transformers for image recognition at scale*, 2020. arXiv: 2010.11929 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2010.11929>.
- [31] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *NeurIPS*, 2017. [Online]. Available: <https://arxiv.org/pdf/1706.03762.pdf>.
- [32] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving language understanding by generative pretraining,” *OpenAI Blog*, 2018. [Online]. Available: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pretraining of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the ACL: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186. [Online]. Available: <https://doi.org/10.18653/v1/N19-1423>.
- [34] C. Raffel, N. Shazeer, A. Roberts, *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *JMLR*, vol. 21, no. 140, 2020.
- [35] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *arXiv preprint arXiv:2101.03961*, 2021. [Online]. Available: <https://arxiv.org/abs/2101.03961>.
- [36] L. Ouyang, J. Wu, X. Jiang, *et al.*, “Training language models to follow instructions with human feedback,” *arXiv preprint arXiv:2203.02155*, 2022. [Online]. Available: <https://arxiv.org/abs/2203.02155>.
- [37] S. Sabour, N. Frosst, and G. E. Hinton, “Dynamic routing between capsules,” *Neural Information Processing Systems (NeurIPS)*, pp. 3857–3867, 2017. [Online]. Available: <https://arxiv.org/abs/1710.09829>.
- [38] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in *ICML*, 2021.

- [39] M. Caron, H. Touvron, I. Misra, *et al.*, *Emerging properties in self-supervised vision transformers*, 2021. arXiv: 2104.14294 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2104.14294>.
- [40] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, “Dinov2: Learning robust visual features without supervision,” *arXiv preprint arXiv:2304.07193*, 2023.
- [41] Z. Liu, Y. Lin, Y. Cao, *et al.*, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *ICCV*, 2021.
- [42] W. Wang, E. Xie, X. Li, *et al.*, “Pyramid vision transformer: A versatile backbone for dense prediction without convolutions,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2021, pp. 568–578.
- [43] X. Chu, Z. Tian, Y. Wang, *et al.*, “Twins: Revisiting the design of spatial attention in vision transformers,” in *NeurIPS*, 2021.
- [44] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy, “Do vision transformers see like convolutional neural networks?” *NeurIPS*, 2021.
- [45] N. Park and S. Kim, “How do vision transformers work?” *ICLR*, 2022.
- [46] MIT 6.070: Introduction to Computers and Programming, *Big o notation*, 2001. [Online]. Available: https://web.mit.edu/6.070/www/lecture/big_o.pdf.
- [47] freeCodeCamp, *Big o notation: Why it matters and why it doesn't*, Jan. 2020. [Online]. Available: <https://www.freecodecamp.org/news/big-o-notation-why-it-matters-and-why-it-doesnt-1674cfa8a23c/>.
- [48] E. Xie and *et al.*, “Co-scale conv-attentional image transformers,” *arXiv preprint arXiv:2104.06399*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.06399>.
- [49] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” *CVPR*, 2022.
- [50] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” *ECCV*, 2020.
- [51] S. Zheng, J. Lu, H. Zhao, *et al.*, “Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers,” *CVPR*, 2021.
- [52] G. Bertasius, H. Wang, and L. Torresani, “Is space-time attention all you need for video understanding?” In *International Conference on Machine Learning (ICML)*, 2021.
- [53] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lucic, and C. Schmid, “Vivit: A video vision transformer,” *arXiv preprint arXiv:2103.15691*, 2021.
- [54] J. Gui, T. Chen, J. Zhang, *et al.*, *A survey on self-supervised learning: Algorithms, applications, and future trends*, 2024. arXiv: 2301.05712 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2301.05712>.
- [55] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.

- [56] C. Buciluă, R. Caruana, and A. Niculescu-Mizil, “Model compression,” in *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 535–541.
- [57] A. Tarvainen and H. Valpola, “Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results,” *Advances in neural information processing systems*, vol. 30, 2017.
- [58] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 9729–9738.
- [59] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International conference on machine learning*, PmLR, 2020, pp. 1597–1607.
- [60] J.-B. Grill, F. Strub, F. Altché, *et al.*, “Bootstrap your own latent-a new approach to self-supervised learning,” *Advances in neural information processing systems*, vol. 33, pp. 21 271–21 284, 2020.
- [61] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, “Unsupervised learning of visual features by contrasting cluster assignments,” *Advances in neural information processing systems*, vol. 33, pp. 9912–9924, 2020.
- [62] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 16 000–16 009.
- [63] J. Li, D. Chen, X. Qi, *et al.*, “Label-efficient learning in agriculture: A comprehensive review,” *Computers and Electronics in Agriculture*, vol. 215, p. 108 412, 2023.
- [64] R. Singh, R. Bidese, K. Dhakal, and S. Sornapudi, “Few-shot adaptation of grounding dino for agricultural domain,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 5332–5342.
- [65] X. Wanyan, S. Seneviratne, S. Shen, and M. Kirley, *Dino-mc: Self-supervised contrastive learning for remote sensing imagery with multi-sized local crops*, 2023. arXiv: 2303.06670 [cs.CV].
- [66] J. Yan, X. Gu, and Y. Chen, “Cropsts: A remote sensing foundation model for cropland classification with decoupled spatiotemporal attention,” *Remote Sensing*, vol. 17, no. 14, p. 2481, 2025.
- [67] C. Bai, L. Zhang, L. Gao, L. Peng, P. Li, and L. Yang, “Dinov2-fcs: A model for fruit leaf disease classification and severity prediction,” *Frontiers in Plant Science*, vol. 15, p. 1 475 282, 2024.
- [68] S. Kar, K. Nagasubramanian, D. Elango, *et al.*, “Self-supervised learning improves agricultural pest classification,” in *AI for Agriculture and Food Systems*, 2021.
- [69] J. Howard and S. Ruder, *Universal language model fine-tuning for text classification*, 2018. arXiv: 1801.06146 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1801.06146>.
- [70] O. Siméoni, H. V. Vo, M. Seitzer, *et al.*, “Dinov3,” *arXiv preprint arXiv:2508.10104*, 2025.

9 Declaration of Originality

The undersigned hereby declares that I

- wrote the work in question independently and without the help of any third party,
- have provided all sources and cited all literature used,
- will protect the confidentiality interests of the client and respect the copyright regulations of the Lucerne University of Applied Sciences and Arts,
- used generative AI only for language correction and sentence refinement; no content was generated or cited from AI tools.

Luzern, 26. November 2025

Sarp Koç

